Автономная некоммерческая организация высшего образования
«Университет Иннополис»
(АНО ВО «Университет Иннополис»)

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(МАГИСТЕРСКАЯ ДИССЕРТАЦИЯ)
по направлению подготовки
09.04.01 Информатика и вычислительная техника

FINAL QUALIFICATION WORK
(MASTER'S THESIS)
Academic Program

09.04.01 Computer Science

Направленность (профиль) образовательной программы
«Искусственный интеллект и инженерия данных»
Field of Study:
«Artificial Intelligence and Data Engineering»

Тема	**Сравнительное исследование алгоритмов обработки сигналов и машинного обучения для обнаружения фибрилляции предсердий при многосуточном мониторировании ЭКГ**
Topic	**A Comparative Study of Signal Processing and Machine Learning Algorithms for Atrial Fibrillation Detection in Long-Term ECG Monitoring**

Работу выполнили / *Prepared by*	**Афончиков Данил Олегович / Danil Afonchikov**	подпись / signature
	Давид Дэниэл Чуквуемека / David Daniel Chukwuemeka	подпись / signature
Руководитель выпускной квалификационной работы / *Final Qualification Work Supervisor*	**Лукманов Рустам Абубакирович / Rustam Lukmanov**	подпись / signature

Иннополис, Innopolis, 2026

Contents

Abstract **9**

1 Introduction **11**
1.1 Background and Problem Statement 11
1.2 Research Gap and Novelty 12
1.3 Research Purpose, Questions, and Objectives 13
1.4 Experimental Approach and Thesis Structure 13
1.5 Significance and Expected Contribution 14

2 Literature Review and Related Work **15**
2.1 Preamble 15
2.2 Evolution of Atrial Fibrillation Detection Methods 16
2.2.1 Classical Digital Signal Processing Approaches 16
2.2.2 Traditional Machine Learning Methods 17
2.2.3 Deep Learning Architectures 17
2.2.4 Transfer Learning and Predictive Approaches 18
2.3 Signal Processing Fundamentals for ECG Analysis 19
2.3.1 Noise Removal and Signal Quality Enhancement 19
2.3.2 QRS Complex Detection 20
2.3.3 P-wave and T-wave Detection 21
2.4 Hybrid DSP-ML Architectures and Computational Efficiency . . . 21
2.4.1 Rationale for Hybrid Approaches 21
2.4.2 Hybrid Architecture Designs 22
2.4.3 Attention Mechanisms and Efficiency Optimization 22
2.4.4 Lightweight Architectures for Wearable Deployment . . . 23
2.4.5 Comparative Studies and Performance Trade-offs 23
2.5 Benchmark Databases and Evaluation Frameworks 25

2.5.1 PhysioNet Standard Databases . . . 25
2.5.2 Challenge Databases and International Contributions . . . 25
2.5.3 Cross-Database Validation and Generalization Challenges 26
2.5.4 Evaluation Methodology Considerations . . . 27
2.6 Long-Term Monitoring and Data-Scale Motivation . . . 27
2.6.1 Extended Duration Monitoring Devices . . . 27
2.6.2 Wearable Consumer Devices . . . 28
2.6.3 Data Volume and Processing Challenges . . . 28
2.6.4 Cloud-Based Architecture Patterns . . . 28
2.6.5 Edge Computing and Real-Time Processing . . . 29
2.6.6 Clinical Workflow Integration . . . 29
2.7 Synthesis and Research Gap . . . 29

3 Framework Design and Methodology 32
3.1 Research Framework and Design Philosophy . . . 33
3.1.1 Summary of Key Design Decisions . . . 34
3.2 Dataset Curation and Selection . . . 35
3.3 Data Pipeline Design . . . 36
3.3.1 Overall Architecture . . . 36
3.3.2 Sampling Rate Standardisation . . . 38
3.3.3 Annotation Normalisation . . . 38
3.3.4 Processed Waveform Serialisation Format . . . 40
3.4 Benchmark Package Design . . . 43
3.4.1 Declarative Experiment Specification . . . 45
3.4.2 Cross-Dataset Experiment Design . . . 45
3.4.3 Split Design . . . 45
3.4.4 Windowing and Label Policy . . . 46
3.4.5 Single-Lead Input Policy . . . 48
3.4.6 Feature and Window Generation . . . 48
3.4.7 Input Representations and Model Contract . . . 48
3.4.8 Reuse and Caching . . . 50
3.4.9 Artefact and Provenance Requirements . . . 50
3.5 Evaluation Methodology . . . 51
3.5.1 Window-Level Metrics . . . 51
3.5.2 Recording-Level Aggregation . . . 52

3.5.3 Computational Cost 53
3.5.4 Reproducibility and Extensibility Criteria 54
3.6 Chapter Summary 54

4 Framework Implementation and Demonstration 55
4.1 Data Pipeline Implementation 56
4.1.1 Implemented Corpus 56
4.1.2 Download and Extract 56
4.1.3 Read, Parse, Resample, and Write 58
4.1.4 Metadata Registration 60
4.2 Benchmark Package Implementation 61
4.2.1 Configuration Validation and Run Identity 62
4.2.2 Command-Line Workflow 63
4.2.3 Deterministic Splits and Conservative Labels 64
4.2.4 Feature and Waveform Window Outputs 66
4.3 Model Plugins and Evaluation Outputs 67
4.3.1 Model Plugin Interface 67
4.3.2 Implemented Model Set 69
4.3.3 Evaluation Metrics and Aggregation 70
4.4 Artefact Storage and Reproducibility 71
4.5 Dashboard and Configuration Builder 73
4.6 Local Infrastructure and Services 77
4.7 Automated Test Coverage 78
4.8 Chapter Summary 80

5 Results, Discussion, and Practical Implications 81
5.1 Demonstration Run and Reported Metrics 81
5.2 Interpretation 84
5.3 Practical Value and Intended Use 84
5.4 Limitations on Interpreting the Current Results 86
5.5 Chapter Summary 87

6 Conclusion and Future Work 88
6.1 Summary of Contributions 88
6.2 Answer to the Research Question 89

6.3 Future Improvements 89
6.4 Conceptual Mock-ups for Future Updates 90
6.5 Final Remarks 91

A Candidate Dataset Survey **92**
A.1 Datasets Not Selected for the Current Implementation 92
A.2 Datasets Retained as Candidates or References 93

B Artefact Compatibility Notes **95**
B.1 Canonical Baseline Configuration 95
B.2 Metric and Matrix Compatibility Aliases 97
B.3 S3 Object Metadata Fields 98
B.4 Legacy Run Identifier Format 98

List of Tables

2.1 Representative reported AF-detection results and the heterogeneous conditions under which they were obtained. 30

3.1 Key methodological design decisions, their defaults, configurable alternatives, and rationale. 34
3.2 Rate-conversion strategies considered for ECG sampling-rate standardisation. 39
3.3 Design contract for benchmark model plugins. 49

4.1 Implemented datasets used to test source-provider access and file-parsing capabilities. 56
4.2 Implemented `BenchmarkModel` interface. 68
4.3 Implemented model plugins. † Rule-based: `requires_training = False`. 69
4.4 Default local port assignments for the Docker Compose stack. . . 78
4.5 Automated test modules in `py_ecg_benchmark_pkg/tests/`. . . 79

5.1 Within-dataset window-level results (mean across implemented datasets) at threshold 0.5. † Trained for 10 epochs, $\leq$4 096 windows, CPU only. 82
5.2 Within- vs. cross-dataset AUROC by model family (mean across all source/target pairs). 82
5.3 Within- vs. cross-dataset AUROC by model (mean across all source/target pairs). 83
5.4 Training time and per-window inference latency by model. 83
5.5 Intended users of the framework and expected benefits. 85

6.1 Conceptual mock-ups for future benchmark updates. 90

List of Figures

3.1 Design overview of the ECG-AF benchmarking framework. . . . 33
3.2 Data-pipeline design from curated manifests and raw downloads to standardised waveforms, annotation sidecars, and metadata registration. 37
3.3 Benchmark package architecture design, showing the main workflow from declarative configuration and processed ECG inputs through split generation, windowing, model execution, evaluation, and persisted artefacts. 44
3.4 Cross-dataset evaluation matrix for one model. Each trained source instance is evaluated on every target dataset; diagonal cells are within-dataset results and off-diagonal cells are cross-dataset transfer results. 46
3.5 Conservative AF-vs-NSR window labelling: only windows fully contained in a single accepted rhythm interval are retained for the primary supervised benchmark. 47

4.1 Canonical benchmark run artefact hierarchy in S3/MinIO. 72
4.2 Dashboard Results view for a completed benchmark run, combining metric tables with timing-against-quality scatter plots to expose the performance–cost trade-off across model families. 74
4.3 Dashboard Explorer view querying the PostgreSQL catalogue to display registered datasets, recording counts, AF-label availability, and the live model registry. 75
4.4 Dashboard Config Builder translating dataset, algorithm, and evaluation selections into a validated YAML configuration ready for export or upload to the artefact store. 75

4.5 Dashboard Run Experiment view for selecting a benchmark configuration, launching an experiment job, and monitoring execution status and logs from the browser. 76
4.6 Dashboard Cache view listing benchmark split and feature cache entries with their cache keys, estimated storage usage, and controls for selective invalidation. 77

Abstract

Atrial fibrillation (AF) is the most common sustained cardiac arrhythmia and a major risk factor for stroke and heart failure, yet its frequently paroxysmal and asymptomatic nature makes detection dependent on long-term ambulatory ECG monitoring that can span days to weeks and generate hundreds of millions of samples per patient. A large body of automated AF detectors has been published with high reported accuracies, but those figures are obtained on different databases, under different train–test splits, label definitions, and metrics, leaving them incomparable and offering little guidance on which methods stay reliable when the data source, recording duration, or computational budget changes.

This thesis treats that gap as an infrastructure problem and presents a reproducible benchmarking framework for AF detection in long-term ECG; its major contribution is the experimental environment in which detectors are compared. The framework comprises a manifest-driven data pipeline that ingests heterogeneous public datasets and standardises them to a common sampling rate, storage format, and conservative AF-versus-normal label policy while preserving provenance; a benchmark harness that evaluates nine registered detector plugins — spanning classical RR-irregularity rules, tabular machine-learning models, and raw-waveform neural networks — behind a shared probability-output interface; and a cross-dataset evaluation matrix that reports window- and recording-level performance alongside computational cost, exposing how detectors generalise across recording sources.

Every experimental decision — dataset manifests, deterministic splits, resolved configurations, metrics, and output artefacts — is stored in a durable, hash-linked form and surfaced through an interactive dashboard, so any reported result is traceable to the exact configuration that produced it. Several demonstration run across the implemented datasets confirms that the framework executes end-to-end and yields fair, traceable, and cost-aware comparisons. The framework provides

the reproducible comparison layer that precedes any clinical ranking of detectors, giving researchers a principled basis for deciding which methods merit the more demanding validation that real deployment requires.

Chapter 1

Introduction

1.1 Background and Problem Statement

Electrocardiogram (ECG) monitoring is a fundamental approach for assessing cardiac activity. It is essential to verify the presence of a normal sinus rhythm and the absence of anomalies during the electrical cycle of the heart. Atrial fibrillation (AF) is one of the most prevalent cardiac arrhythmias, affecting approximately 2% of the general population [1]. This condition is both common and clinically serious: patients with sustained AF face significantly higher risks of life-threatening complications, including heart failure, stroke, and sudden death [1].

Long-term longitudinal monitoring, facilitated by devices such as Holter monitors and ECG patches, is the standard method for detecting episodic AF. These studies can last from 24 hours to several months, generating large amounts of continuous data. The subsequent analysis of these multi-day recordings demands substantial manual effort from cardiologists and technicians, making the process time-consuming, costly, and prone to human error. This manual analysis bottleneck represents a significant problem in modern cardiology.

Advances in digital signal processing (DSP) and machine learning (ML) offer a promising solution. These technologies can automatically analyze lengthy ECG recordings, identify segments of interest, and flag potential AF episodes, thereby streamlining the clinician's workflow. Several open algorithms exist for AF detection; however, their performance and accuracy are often measured under specific, constrained conditions. Their efficacy, computational efficiency, and reliability remain unverified for large-scale general clinical practice, where signal

quality and patient physiology vary widely.

1.2 Research Gap and Novelty

Although previous research has developed many AF detection algorithms, reproducible comparison across datasets remains difficult. Published studies often differ in their data sources, preprocessing choices, label definitions, train–test splits, evaluation metrics, and reporting practices. In addition, ECG recordings are sensitive medical data, so relatively few datasets are openly available for unrestricted general use. Privacy, consent, licensing, and institutional governance constraints often limit data sharing, which makes transparent use of the available datasets especially important. As a result, it is often unclear whether performance differences are caused by the algorithms themselves or by differences in the experimental pipeline. This gap is especially important for long-term ECG monitoring, where recordings may contain large volumes of data, heterogeneous signal quality, and annotations produced under different clinical or technical assumptions.

This study's contribution is a reproducible data pipeline and benchmarking framework that makes AF detection experiments easier to configure, repeat, inspect, and extend. Its scope is the experimental infrastructure for comparing detectors, not the design of a new detection algorithm or a clinical monitoring service. The main novel contributions are:

1. A manifest-driven data pipeline that converts heterogeneous ECG recordings and annotations into a consistent, benchmark-ready corpus while preserving dataset provenance and processing decisions.

2. A standardized benchmarking framework that supports deterministic split generation, configurable label policies, a common model interface, and shared evaluation metrics for fair comparison of AF detection methods.

3. A reproducible experiment workflow that stores configurations, outputs, metrics, and artefacts in a structured form, enabling researchers to inspect results, rerun experiments, and extend the framework with additional datasets or algorithms.

1.3 Research Purpose, Questions, and Objectives

The overall purpose of this research is to design, implement, and evaluate a reproducible data pipeline and benchmarking framework for atrial fibrillation detection experiments in long-term ECG recordings. The work focuses on the experimental infrastructure needed to prepare data consistently, run comparable evaluations, and preserve the information required to reproduce results.

This purpose is driven by the following primary research question: *How can a reproducible data pipeline and benchmarking framework be designed to support fair, scalable, and transparent comparison of atrial fibrillation detection methods in long-term ECG monitoring?*

To answer this question, the study defines the following specific objectives:

1. To identify the experimental decisions that make published AF-detection results difficult to compare across datasets.

2. To specify reproducible preparation, labelling, splitting, and evaluation procedures that can be applied consistently to heterogeneous ECG sources.

3. To implement those procedures as a working benchmark workflow that can run multiple detector families under the same rules.

4. To assess whether the implemented workflow produces traceable outputs, reusable artefacts, and practical cross-dataset comparisons on selected ECG corpora.

1.4 Experimental Approach and Thesis Structure

The experimental approach follows the same logic as the research gap. The thesis first reviews the technical and methodological background for AF detection, then designs a reproducible benchmarking methodology, implements it as a working software artefact, and finally discusses how the artefact can be used in real research and pre-clinical evaluation. The final chapters are therefore separated deliberately: Chapter 5 interprets the demonstrated results and practical implications, while Chapter 6 closes the thesis by summarising the contribution and outlining future improvements.

The structure of this thesis is as follows:

Chapter 2: Literature Review and Related Work Provides a comprehensive overview of existing AF detection methods, long-term ECG monitoring technologies, and previous work in algorithm benchmarking.

Chapter 3: Framework Design and Methodology Details the design of the data pipeline, benchmark-ready corpus representation, label policies, split generation, model interface, and evaluation workflow.

Chapter 4: Framework Implementation and Demonstration Presents the implemented data pipeline, benchmark package, artefact storage, dashboard, and end-to-end demonstration workflow.

Chapter 5: Results, Discussion, and Practical Implications Interprets the demonstration results, explains the real-life value of the benchmark, identifies intended users, and discusses current limitations.

Chapter 6: Conclusion and Future Work Summarizes the thesis contributions, answers the research question, and describes future improvements and conceptual mock-ups for extending the benchmark.

The appendices provide supporting material: a candidate dataset survey and compatibility notes for the implementation artefacts.

1.5 Significance and Expected Contribution

The expected contribution of this study is the reproducible framework introduced in Section 1.2; this section explains why that contribution matters. Scientifically, the thesis provides a methodology for making AF algorithm comparisons more transparent and reproducible across heterogeneous ECG sources. Practically, it offers an extensible software artefact for inspecting how data were prepared, how splits and labels were derived, and how metrics and artefacts were produced.

Chapter 2

Literature Review and Related Work

2.1 Preamble

This chapter reviews the current state of knowledge in atrial fibrillation (AF) detection through electrocardiogram (ECG) analysis, with particular emphasis on signal processing algorithms, machine learning approaches, benchmark datasets, and reproducible evaluation practices. The review addresses the central research question of how a data pipeline and benchmarking framework can support fair, scalable, and transparent comparison of AF detection methods in long-term ECG monitoring.

The literature is organized thematically into six major areas. Section 2.2 examines the evolution of AF detection methods from classical signal processing through traditional machine learning to contemporary deep learning approaches. Section 2.3 reviews signal processing fundamentals including noise removal, R-peak detection, and ECG component delineation that form the preprocessing foundation for detection algorithms. Section 2.4 analyzes hybrid architectures that combine domain-specific preprocessing with data-driven classification, with particular attention to computational efficiency considerations. Section 2.5 evaluates benchmark databases and comparative studies that establish performance baselines and reveal generalization challenges. Section 2.6 examines long-term monitoring technologies primarily as sources of large, heterogeneous ECG data and as motivation for scalable, reproducible benchmarking. Section 2.7 synthesizes the review,

identifies the research gap created by non-comparable evaluation conditions, and justifies the methodological approach adopted in this thesis.

2.2 Evolution of Atrial Fibrillation Detection Methods

2.2.1 Classical Digital Signal Processing Approaches

The detection of atrial fibrillation through automated ECG analysis has been investigated for several decades. Early approaches focused primarily on quantifying the characteristic RR interval irregularity that distinguishes AF from normal sinus rhythm. Park et al. [1] introduced the RdR map method, which plots RR intervals against their successive differences using a 25-millisecond resolution grid. Their validation on the MIT-BIH Atrial Fibrillation Database demonstrated sensitivity and specificity exceeding 90% using only 32–64 cardiac cycles. This approach remains relevant because of its minimal computational requirements, making it suitable for implantable cardiac devices with severe power constraints.

Dash et al. [2] developed an automatic real-time detection method combining three complementary features: root mean square of successive differences (RMSSD), Shannon entropy, and autocorrelation. Their three-step pipeline achieved up to 99.5% accuracy on 24-hour recordings. These foundational methods established that RR interval irregularity combined with P-wave morphology analysis provides robust AF discrimination.

Heart rate variability (HRV) analysis has expanded the feature space for AF detection. Researchers have employed time-domain measures including SDNN (standard deviation of NN intervals), RMSSD, and pNN50 (percentage of successive RR intervals differing by more than 50 ms), frequency-domain decomposition examining LF/HF ratio (low frequency to high frequency power), and nonlinear indices including sample entropy and Poincaré plot parameters. The 2017 PhysioNet/Computing in Cardiology Challenge demonstrated that combining statistical, geometrical, frequency, entropy, and Lorentz plot-based features with support vector machines yielded competitive classification performance on short single-lead recordings [3].

2.2.2 Traditional Machine Learning Methods

The application of traditional machine learning classifiers to AF detection has been systematically compared in recent literature. Jahan et al. [4] evaluated seven classifiers including CART, KNN, C4.5, ID3, SVM, Random Forest, and Rotation Forest on the PhysioNet Challenge 2017 dataset comprising 5,154 normal recordings, 771 AF recordings, and 2,557 other rhythms. They found that Random Forest achieved 97% accuracy and that ensemble methods consistently outperformed individual classifiers.

Xie et al. [5] conducted rigorous external validation using leave-one-group-out cross-validation across three databases: the Long-Term AF Database, MIT-BIH Arrhythmia Database, and MIT-BIH Normal Sinus Rhythm Database. Their study demonstrated that AdaBoost showed superior generalization with 85.67% sensitivity, 81.25% specificity, and 88.18% F1-score using only 20 heartbeats, highlighting the importance of cross-database validation for assessing algorithm robustness.

Feature engineering for traditional machine learning typically combines morphological descriptors, statistical measures, and transform-domain representations. Wavelet coefficients, time-frequency domain features, and RR interval variability metrics have proven particularly effective for distinguishing AF from normal sinus rhythm and other arrhythmias.

These figures are reported under different database selections, validation schemes, and label definitions, so they demonstrate feasibility; Section 2.7 explains why they cannot support a reliable classifier ranking.

2.2.3 Deep Learning Architectures

Deep learning has fundamentally altered AF detection capabilities since 2019. Hannun et al. [6] demonstrated that a 34-layer residual convolutional neural network trained on single-lead wearable ECG data could exceed the average cardiologist's F1 score for arrhythmia classification, establishing deep learning as clinically competitive. This landmark study analyzed 91,232 single-lead ECGs from 53,877 patients and compared algorithm performance against board-certified cardiologists.

The systematic review by Xie et al. [7], synthesizing 14 studies, found that

CNN-based deep learning achieved 98.1% sensitivity compared to 91.5% for traditional machine learning, with AUC values of 0.99–1.00 versus 0.97–0.99 respectively. This meta-analysis provides strong evidence that deep learning architectures capture patterns that traditional feature engineering cannot fully specify.

Contemporary architectures have incorporated increasingly sophisticated designs. The MIF-AFNet architecture [8] fuses residual CNN and bidirectional LSTM networks for simultaneous ECG morphology and RR interval analysis. Zou et al. validated this approach through five-fold cross-validation on CPSC2021 with external testing on AFDB, LTAF, MITDB, and NSRDB databases, demonstrating generalization across diverse ECG sources.

ArNet2 [9] represents a landmark achievement in geographic generalization. Biton et al. validated this algorithm across 2,147 patients totaling 51,386 hours from four centers spanning the United States, Israel, Japan, and China. The study demonstrated robustness across demographic variations, though performance decreased for atrial flutter detection, highlighting the importance of training data diversity for developing clinically robust algorithms.

Transformer-based architectures represent the current frontier in AF detection. The Component-Aware Transformer (CAT) [10] tokenizes ECG signals by clinical components, mimicking cardiologist interpretation strategies. MSGformer [11] employs multi-scale grid transformers with self-attention mechanisms for 12-lead analysis, achieving state-of-the-art performance on multiple benchmark databases.

The reported deep-learning results show the strength of learned representations, but their datasets, splits, class definitions, and metrics vary enough that they should not be read as a leaderboard; the comparability problem is synthesized in Section 2.7.

2.2.4 Transfer Learning and Predictive Approaches

Transfer learning addresses the persistent challenge of limited labeled data in medical applications. Mehari and Strodthoff [12] demonstrated that pretraining on the Icentia11K dataset comprising 11,000 patients significantly reduced annotation requirements for fine-tuning on PhysioNet Challenge data. Their approach achieved competitive performance with substantially fewer labeled examples from the target domain.

A Siamese network approach [13] achieved 96.84–97.07% macro-F1 with only 1–11 ECG segments (30–180 seconds) for patient-specific adaptation. This result demonstrates that personalized AF detection models can be developed with minimal patient-specific data when leveraging transfer learning from large-scale pretraining.

The WARN model [14] pioneered predictive capability by forecasting AF onset rather than detecting existing episodes. Choi et al. achieved 83% accuracy in predicting AF with an average prediction horizon of 30.8 minutes using only RR intervals, making this approach suitable for wearable deployment. Predictive AF detection could enable preventive interventions before episodes occur, though clinical validation of this capability remains limited.

2.3 Signal Processing Fundamentals for ECG Analysis

2.3.1 Noise Removal and Signal Quality Enhancement

The quality of AF detection algorithms depends critically on preprocessing that removes noise while preserving diagnostically relevant morphology. ECG signals suffer from multiple noise sources: baseline wander below 0.5 Hz, powerline interference at 50/60 Hz, electromyographic noise at high frequencies, and motion artifacts particularly in ambulatory settings.

Baseline wander removal has been extensively studied using wavelet-based decomposition. A comparative evaluation [15] of 14 wavelets across 5 families found that Daubechies-3 and Symlets-3 wavelets with 7 decomposition levels achieved optimal performance (MSE=0.0044) on the QT Database comprising 105 patients. Singh and Joshi [16] demonstrated that empirical wavelet transform outperforms traditional linear filters and empirical mode decomposition for baseline correction on the MIT-BIH Arrhythmia Database.

Deep learning denoising has emerged as a viable alternative to traditional filtering. DeScoD-ECG [17] applied deep score-based diffusion models to achieve 20% improvement over baseline methods on the QT Database and MIT-BIH Noise Stress Test Database. The biGRU denoiser [18] trained on combined MIT-BIH Noise Stress Test and PTB-XL databases handles electrode motion, muscle activa-

tion, and baseline wander with RMSE of 0.023–0.041, demonstrating the capability of recurrent neural networks to learn denoising functions from data.

Signal quality assessment enables rejection of uninterpretable segments before analysis. The fuzzy comprehensive evaluation method [19] combining multiple signal quality indices with Cauchy, rectangular, and trapezoidal membership functions achieved 97.67% accuracy in three-level quality classification on PhysioNet data. CNN-LSTM hybrid architectures [20] achieve 98.65% F1-score for signal quality assessment, while wearable-specific approaches [21] estimate RR interval accuracy rather than merely detecting noise, achieving 96.74% accuracy for fabric electrode ECG quality classification.

2.3.2 QRS Complex Detection

QRS detection remains the foundation of beat-level ECG analysis. The Pan-Tompkins algorithm [22] established the standard approach using bandpass filtering (5–15 Hz), derivative filtering, squaring, moving average integration, and adaptive thresholding. This classical method achieved 99.3% detection accuracy on the MIT-BIH Arrhythmia Database and has been widely implemented in commercial ECG systems.

Pan-Tompkins++ [23] introduced three-threshold pattern-based adjustment with 5–18 Hz bandpass, reducing false positives by 2.8%, false negatives by 1.8%, and improving execution speed by 33% with publicly available code. The WT-SEE algorithm [24] combining wavelet transform with modified Shannon energy envelope achieved 99.39% sensitivity and 99.49% positive predictivity on MIT-BIH while running 30.6% faster than Pan-Tompkins, demonstrating that careful algorithmic optimization can improve both accuracy and efficiency.

Deep learning QRS detection has achieved the highest reported performance. A feature pyramid network with deep neural network (FPN-DNN) [25] demonstrated 99.88% sensitivity and 99.85% positive predictivity across seven public databases. Zahid et al. [26] developed robust R-peak detection specifically for low-quality Holter ECGs using 1D CNN architectures that handle arrhythmia beats, powerline interference, and baseline wandering. A ResNet-GRU hybrid [27] annotated 619,681 QRS complexes across 12 public databases, demonstrating scalability for large-scale processing.

2.3.3 P-wave and T-wave Detection

P-wave detection presents particular challenges for AF analysis because P-waves are absent during fibrillation, low in amplitude relative to QRS complexes, and highly variable across patients and pathologies. Saclova et al. [28] developed a phasor transform method with pathology-informed decision rules, achieving 98.56% sensitivity and 99.82% positive predictivity for physiological P-waves on MIT-BIH. However, performance degraded on pathological recordings including AF, premature ventricular contractions, and atrioventricular block (93.07% sensitivity, 88.60% positive predictivity), highlighting the difficulty of reliable P-wave detection in abnormal rhythms.

U-Net architectures for ECG delineation [29] handle AF and atrial flutter with classification-guided hybrid loss functions that reduce false P-wave predictions during fibrillatory rhythms. These deep learning approaches achieve greater than 99% F1-score for QRS and T-wave detection and greater than 97% for P-waves on the Lobachevsky University Database, demonstrating that end-to-end learning can handle complex morphological variations that rule-based methods cannot fully capture.

2.4 Hybrid DSP-ML Architectures and Computational Efficiency

2.4.1 Rationale for Hybrid Approaches

The convergence of classical signal processing with machine learning has produced hybrid architectures that leverage domain knowledge for interpretable preprocessing while exploiting deep learning's pattern recognition capabilities. This approach addresses a fundamental tension in clinical AI: pure deep learning achieves highest accuracy but provides limited insight into detection mechanisms, hampering clinical acceptance and regulatory approval. Hybrid architectures enable clinicians to understand algorithm behavior through familiar signal processing operations (filtering, peak detection, feature extraction) while maintaining the flexibility to learn complex patterns through data-driven classification. This balance has motivated substantial recent research into efficient hybrid designs

suitable for resource-constrained deployment.

2.4.2 Hybrid Architecture Designs

The CWT-CNN-LSTM architecture [30] transforms one-dimensional ECG signals into two-dimensional scalogram images at 227x227x3 resolution using continuous wavelet transform, followed by 2D-CNN feature extraction and LSTM classification. Madan et al. achieved 98.7–99% accuracy on MIT-BIH with 6 minutes 30 seconds training time on Intel Xeon Gold processors, demonstrating that time-frequency transformation combined with deep learning can achieve excellent performance.

A two-subsystem hybrid architecture [31] combines ResNet-LSTM deep feature extraction with traditional feature engineering plus Random Forest classification using SMOTE class balancing, achieving 99.26% accuracy on MIT-BIH. This approach explicitly maintains both learned representations and engineered features, enabling interpretation of which feature types contribute most to classification.

The STFT/CWT-CNN-LSTM edge model [32] achieves 99.6% accuracy with 9 milliseconds inference time, 90 KB model size, and 12 MB memory footprint. Han et al. demonstrated that hybrid architectures can be optimized for real-time edge deployment without sacrificing accuracy, addressing a critical requirement for wearable cardiac monitoring systems.

These hybrid studies are useful evidence that engineered preprocessing and learned representations can be combined, but their headline values depend on different datasets, splits, and deployment assumptions rather than on a shared experimental contract; Section 2.7 explains why this prevents direct ranking.

2.4.3 Attention Mechanisms and Efficiency Optimization

Attention mechanisms enhance hybrid architectures by enabling dynamic focus on diagnostically relevant signal segments. The DBA-ASFNet architecture [33] combines depthwise-separable convolutional residual blocks with bidirectional GRU and attention mechanisms, achieving approximately 0.03 million parameters, 6.43 MFLOPs per inference, and 2 milliseconds latency on Raspberry Pi 5. This represents a fundamental advance in deployment-ready efficiency, demonstrating that sophisticated neural architectures can operate on consumer-grade edge

hardware.

The Hybrid Attention-Based Deep Learning Network (HADLN) [34] integrates ResNet with bidirectional LSTM and attention mechanisms for enhanced feature representation across temporal scales. The attention mechanism learns to weight different portions of the ECG signal based on their diagnostic relevance, mimicking the selective attention that cardiologists apply when interpreting recordings.

2.4.4 Lightweight Architectures for Wearable Deployment

Lightweight architectures enable wearable deployment through efficient computational design. ArrhythmiNet [35] adapts MobileNet's depthwise separable convolutions for one-dimensional ECG signals, significantly reducing multiply-accumulate operations while maintaining real-time inference capability. LCG-Net [36] achieves 98.39% accuracy with only 39,040 parameters and 8.16 million computations for clinical AF detection, demonstrating that careful architecture design can dramatically reduce computational requirements.

Model compression techniques further reduce computational requirements. Post-training 8-bit integer quantization [37] achieves 98.5% sensitivity and 99.8% specificity with 7.65 milliseconds inference time and 5.85 millijoules energy per prediction on smartphones. Dynamic range quantization [38] produces models as small as 139 KB achieving 94.6% accuracy for wearable deployment. Unstructured pruning [39] achieves 70% sparsity while maintaining 98.99% accuracy on PTB-XL. Knowledge distillation approaches [40] transfer discriminative capability from large teacher models to compact student networks suitable for edge inference. This technique enables deployment of models that approach the accuracy of large networks while operating within the memory and computational constraints of wearable devices.

2.4.5 Comparative Studies and Performance Trade-offs

Comparative studies reveal important accuracy-efficiency trade-offs. One-dimensional recurrent networks (GRU, LSTM) have been shown to outperform two-dimensional CNNs applied to time-frequency representations for ECG classification [41]. GRU achieved 79.67% sensitivity and 81.04% specificity on PTB-XL

while avoiding the computational overhead of wavelet or Fourier transformation, suggesting that native one-dimensional processing may be preferable for certain deployment scenarios.

ResNet-18, with the third-fewest parameters among ResNet variants, consistently performs best across time-frequency methods [42], indicating that larger models do not necessarily improve ECG classification. This finding suggests diminishing returns for model size increases beyond certain thresholds. Bidirectional LSTM networks achieve 28% fewer parameters than conventional LSTM with higher accuracy through efficient bidirectional context modeling [43].

Scalability for large-scale processing requires parallel computing architectures. Mobile GPU acceleration [44] achieves 5.81x speedup on the Adreno 530 processor for 24-hour ECG analysis, reducing processing time from over 7 seconds to 1.215 seconds with 64% energy consumption reduction through heterogeneous CPU-GPU pipelines. High-performance computing studies [45] demonstrate training on 272 GB of ECG data across 16 datasets using multi-GPU configurations, finding sub-linear scaling (1.6x for 2 GPUs, 1.9x for 4 GPUs) with optimal configuration of 16 CPUs and sliding window size of 128.

These studies show that computational behaviour is not a secondary implementation detail in AF detection. Training duration affects how easily models can be reproduced, tuned, and extended to new datasets, while inference latency affects whether a detector is practical for long-term monitoring, wearable screening, or near-real-time clinical review. Consequently, a fair benchmark should report computational cost alongside classification quality rather than presenting sensitivity, specificity, or AUROC in isolation. This motivates the thesis framework's use of training time, prediction time, and full evaluation time as explicit benchmark outputs, enabling accuracy–cost comparisons across classical rule-based detectors, tabular machine learning models, and raw-waveform neural networks.

2.5 Benchmark Databases and Evaluation Frameworks

2.5.1 PhysioNet Standard Databases

Standardized evaluation requires publicly accessible databases with reliable annotations and consistent evaluation protocols. The PhysioNet repository [46], established by Goldberger et al. in 2000, provides the foundational infrastructure for reproducible AF detection research.

The MIT-BIH Arrhythmia Database (MITDB) comprises 48 half-hour recordings from 47 subjects with approximately 110,000 beat annotations verified by two or more cardiologists, using modified limb lead II and V1 at 360 Hz with 11-bit resolution. Despite its age (recordings from 1975–1979) and limited sample size, MITDB remains the most cited benchmark for arrhythmia detection algorithms. The MIT-BIH AF Database (AFDB) contains 25 recordings of 10 hours each at 250 Hz with manual rhythm annotations distinguishing AF, atrial flutter, AV junctional rhythm, and other rhythms. The Long-Term AF Database (LTAFDB) provides 84 recordings of 24–25 hours at 128 Hz with over 9 million annotations by MEDICALgorithmics Ltd using FDA-approved analysis systems.

PTB-XL [47] represents the largest freely accessible clinical 12-lead ECG dataset with 21,837 recordings from 18,869 patients at both 500 Hz and 100 Hz sampling rates, annotated with 71 SCP-ECG statements by up to two cardiologists. Wagner et al. established PTB-XL as a comprehensive benchmark for multi-label ECG classification. The accompanying benchmark framework [48] provides standardized 10-fold cross-validation splits, seven multi-label prediction tasks, recommended macro-averaged AUROC metrics, and a GitHub repository for reproducible model comparison.

2.5.2 Challenge Databases and International Contributions

The China Physiological Signal Challenge databases contribute significant diversity. CPSC 2018 includes 6,877 recordings from 11 Chinese hospitals with 12-lead ECG at 500 Hz spanning 6–60 seconds, annotated for nine classes including AF with multi-label possibility. CPSC 2021 focuses specifically on paroxysmal AF event detection with novel episode-level scoring metrics that better reflect clinical

detection requirements.

The Computing in Cardiology Challenge 2017 provided approximately 10,000 single-lead recordings from AliveCor devices at 300 Hz in four classes (Normal, AF, Other, Noisy), establishing a benchmark for smartphone-based AF detection. Challenge 2020 [49] aggregated 66,361 recordings from six sources across four countries for standardized multi-database evaluation of 27 diagnoses, representing the most comprehensive multi-center challenge dataset assembled for ECG classification.

Newer databases address geographic diversity limitations. IRIDIA-AF [50] provides 167 records from 152 Belgian patients with 19–95 hours per record at 200 Hz, totaling 24 million seconds of expert-annotated Holter monitoring. SHDB-AF [51] contributes the first Japanese AF Holter database with 128 recordings from 122 patients comprising 21.6 million seconds, validated with ArNet2 (F1=0.92) and RawECGNet (F1=0.93). These databases expand geographic diversity and enable assessment of cross-population generalization.

2.5.3 Cross-Database Validation and Generalization Challenges

Cross-database validation studies reveal concerning performance degradation. Kim et al. [52] trained ResNet models on individual databases achieving 98–99% internal accuracy but only 53–92% external accuracy, demonstrating severe data dependency in deep learning AF detection. This finding highlights a critical limitation: algorithms that achieve excellent performance on their training database often fail to generalize to independent patient populations with different recording equipment, electrode configurations, and demographic characteristics.

The PhysioNet Challenge 2020 evaluation [49] found that algorithm performance varied significantly by source database, with Chinese Physiological Signal Challenge data (most balanced class distribution) showing best performance while highlighting equipment and population heterogeneity challenges. AF-DETR [53] achieved heartbeat-level F1 of 90.55–99.87% and segment-level accuracy of 97.30–99.99% across five databases (CPSC2021, AFDB, LTAFDB, MITDB, NSRDB), representing strong cross-database generalization through transformer-based architecture.

2.5.4 Evaluation Methodology Considerations

Evaluation methodology inconsistencies complicate literature comparison. Studies employ sample-based metrics (evaluating each time point), episode-based metrics (evaluating complete AF events), and segment-based metrics (evaluating fixed-duration windows) with varying definitions of minimum episode duration, overlap between windows, and class imbalance handling. The absence of consensus standards for train-test splits enables potential data leakage and overfitting to specific databases.

AAMI-compliant evaluation protocols [54] remain underutilized despite established standards for medical device validation. The AAMI EC57 standard specifies requirements for testing and reporting performance of automated ECG analysis systems, including minimum database sizes, required arrhythmia types, and performance metrics. Adherence to these standards would improve comparability across published studies.

2.6 Long-Term Monitoring and Data-Scale Motivation

2.6.1 Extended Duration Monitoring Devices

The clinical imperative for AF detection in long-term recordings stems from the paroxysmal nature of many AF episodes. Traditional 24-hour Holter monitoring achieves only 3–5% diagnostic yield for AF in post-stroke patients [55], while extended monitoring increases the chance of observing intermittent episodes.

Extended Holter and patch studies demonstrate this duration effect. The Find-AF study [56] found that 72-hour Holter detected unknown AF in 12.5% of stroke patients compared with 4.8% in the first 24 hours, while a 14-day patch comparison [57] reported substantially higher arrhythmia and AF/atrial-flutter detection than 24-hour Holter. ECG patch studies and trials, including Zio patch reviews [58] and GUARD-AF [59], show that multi-day single-lead monitoring is now a substantial source of ECG evidence. What matters for this thesis is that longer recordings create large, heterogeneous data volumes that motivate a scalable, reproducible benchmark.

2.6.2 Wearable Consumer Devices

Consumer wearables extend AF screening and monitoring beyond traditional clinical devices, but their relevance here is mainly as a source of large single-lead or pulse-derived datasets with variable adherence, device behaviour, and signal quality. Large studies such as the Apple Heart Study [60] illustrate the scale of these data streams; they also reinforce why benchmark pipelines must record data source, sensing modality, and quality assumptions rather than treating all long-term recordings as interchangeable.

2.6.3 Data Volume and Processing Challenges

Data volume and analysis challenges scale with monitoring duration. Sampling at 360 Hz generates approximately 100–200 MB per 24-hour recording, with 14-day patches producing multiple gigabytes per patient. Long-term monitoring of large patient populations can generate terabytes of data requiring efficient storage, transmission, and processing infrastructure.

Signal quality degrades in ambulatory settings due to motion artifacts, electrode impedance changes, and muscle noise, with inconclusive rates of 17–26% across smartwatch studies [61]. Battery life constraints limit continuous transmission: ECG patches achieve 7–14 days single use, while smartwatches require daily charging (18–24 hour battery life typical). These scale, heterogeneity, and quality issues motivate the pipeline design used in this thesis: records are standardised through explicit manifests and adapters, serialised into efficient Parquet outputs, and prepared for reusable Spark-based feature generation.

2.6.4 Cloud-Based Architecture Patterns

Cloud ECG systems often distribute acquisition, mobile gateway, storage, analysis, and visualisation responsibilities across multiple services [62]. This thesis does not implement an enterprise cloud architecture; its local Docker Compose stack is used only to make storage, databases, and dashboards reproducible during benchmarking. The relevant lesson is therefore architectural separation of concerns, not clinical cloud deployment.

2.6.5 Edge Computing and Real-Time Processing

Edge computing is relevant because long-term monitoring systems may eventually need algorithms that are both accurate and inexpensive to run on constrained hardware. The benchmark therefore records training and inference timing so that computational cost can be compared alongside accuracy. Device-level latency catalogues and real-time alerting requirements are treated as deployment concerns outside the implemented framework.

2.6.6 Clinical Workflow Integration

Clinical workflow integration is outside the scope of this thesis; the framework supports pre-deployment comparison and audit of AF detectors rather than integration with electronic health records or commercial monitoring operations.

2.7 Synthesis and Research Gap

The literature shows clear technical maturation in automated AF detection. Classical digital signal processing methods established that RR irregularity, HRV features, and P-wave information can separate AF from normal rhythm. Traditional machine learning then expanded this foundation through engineered morphology, rhythm, and transform-domain features, while deep learning and transformer architectures have learned increasingly complex representations directly from ECG signals. Across these families, many studies report high headline accuracies, sensitivities, specificities, F1 scores, or AUROC values.

However, the reported values in Sections 2.2–2.4 are not directly comparable. They are obtained under different datasets, train–test split conventions, label definitions such as AF-vs-NSR versus AF-vs-all or episode-level versus window-level targets, metrics such as accuracy, F1, AUROC, sensitivity, specificity, and episode-level scores, and different signal-quality handling policies. The tension is already visible in Section 2.5: cross-database validation work shows that high internal scores may degrade substantially when the evaluation database changes, and evaluation-methodology studies note inconsistent sample-, segment-, and episode-based scoring.

The spread of datasets, splits, label definitions, and metrics across Table 2.1

Table 2.1: Representative reported AF-detection results and the heterogeneous conditions under which they were obtained.

Study (citation)	**Dataset(s)**	**Split / validation**	**Positive-class definition**	**Headline metric & value**
Hannun et al. [6]	Single-lead wearable ECGs	Cardiologist comparison on ambulatory ECGs	Multiclass rhythm classification including AF	Average F1 0.837 versus cardiologists 0.780
Jahan et al. [4]	PhysioNet Challenge 2017	Classifier comparison on the challenge dataset	AF among normal, AF, and other rhythms	Random Forest 97% accuracy
Xie et al. [5]	LTAFDB, MITDB, and NSRDB	Leave-one-group-out cross-database validation	AF detection from 20 heartbeats	AdaBoost 85.67% sensitivity, 81.25% specificity, 88.18% F1
Xie et al. [7]	Fourteen studies in meta-analysis	Systematic-review synthesis	AF detection across reviewed ECG studies	CNN 98.1% sensitivity versus ML 91.5%
Zou et al. [8]	CPSC2021 plus AFDB, LTAF, MITDB, and NSRDB	Five-fold CPSC2021 validation with external testing	Binary AF detection using ECG and RR information	98.63% CPSC2021 accuracy; 97.04–100% external accuracy
Biton et al. [9]	Multi-country data from the United States, Israel, Japan, and China	External geographic validation across four centres	AF/atrial-flutter versus non-AF rhythm windows	ArNet2 F1 0.90–0.95 across geography
Kim et al. [52]	Multiple AF-related ECG databases	Internal versus external database testing	AF detection using ResNet models	98–99% internal accuracy versus 53–92% external accuracy

is precisely what prevents a fair ranking from the literature alone; Kim et al. [52] provide the strongest motivating example, showing that the same ResNet-style architecture can move from 98–99% internal accuracy to 53–92% external accuracy when the evaluation database changes.

The resulting research gap is not simply the absence of another detector. Rather, the literature lacks a shared experimental contract that fixes how data are prepared, how labels are derived, how splits are generated, how models exchange predictions with the evaluator, how metrics are computed, and how artefacts are preserved. Without that contract, it remains unclear whether differences between reported results reflect algorithmic improvement or differences in the surrounding pipeline. Reproducible, transparent, cross-dataset comparison is therefore missing.

This thesis responds to that gap by treating reproducible AF benchmarking as the main contribution: a data pipeline plus benchmarking framework that standardises heterogeneous ECG inputs, records provenance, supports deterministic and configurable experiments, evaluates models through a shared interface, and preserves outputs for inspection. Chapter 3 translates this literature gap into the concrete methodological design of the framework.

Chapter 3

Framework Design and Methodology

This chapter presents the architecture and methodological design of the project. It explains how the data pipeline and benchmark workflow were planned, which design decisions guided the framework, and how those decisions provide a holistic and easily extendable structure for atrial fibrillation (AF) detection research in electrocardiography (ECG). To make this design concrete, the chapter specifies the components that later become implementation artefacts: dataset curation, data standardisation, annotation handling, splitting, labelling, feature generation, model-interface design, cross-dataset evaluation, and provenance. Chapter 4 then moves from architecture to implementation, describing the development cycle, realised software and machine-learning components, and results produced by the current framework.

The methodology is organised around two connected layers. The first layer is a data-preparation design that turns heterogeneous ECG datasets into a consistent benchmark-ready corpus while preserving provenance. The second layer is an experiment-design layer that defines how recordings are split, windowed, labelled, scored across datasets, and documented. Figure 3.1 summarises the intended end-to-end architecture.

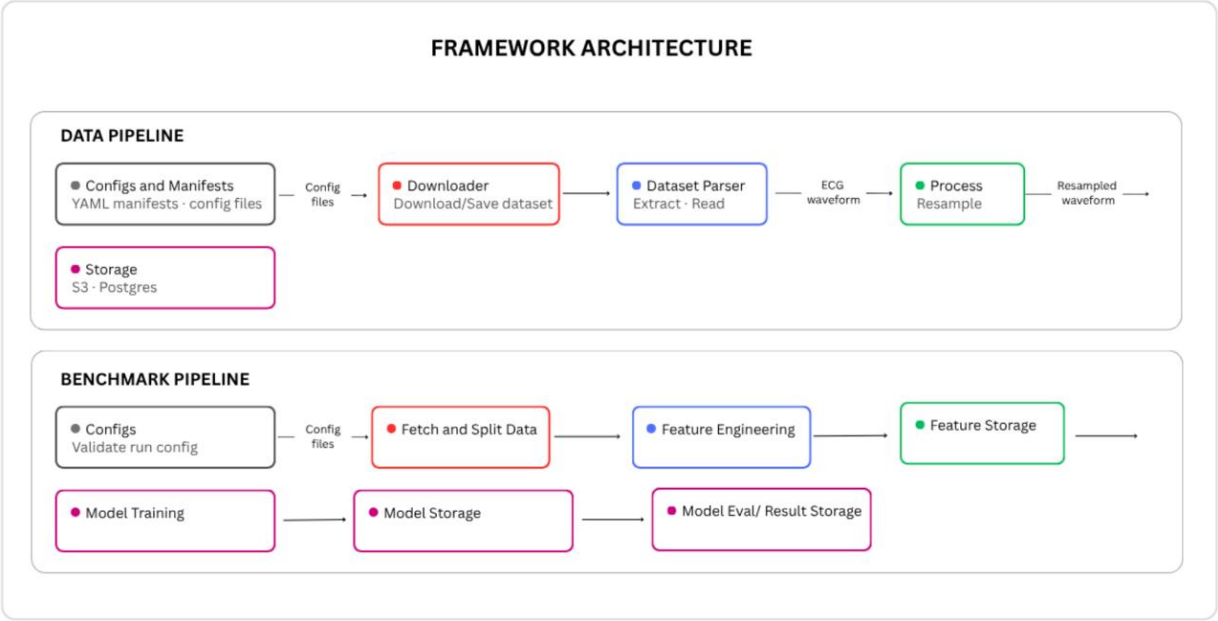


Figure 3.1: Design overview of the ECG-AF benchmarking framework.

3.1 Research Framework and Design Philosophy

The central methodological problem addressed by this thesis is that AF detection algorithms are often reported under study-specific assumptions about preprocessing, dataset selection, labelling, train–test splitting, threshold selection, and metric calculation. When those assumptions are not expressed as reusable experimental objects, it becomes difficult to determine whether a reported performance difference is caused by the detector itself or by differences in data handling. A related and often under-reported issue is *generalisation*: a detector tuned and tested on a single dataset may not maintain its performance when applied to recordings from a different population, device, or annotation protocol. The framework is therefore designed to make data preparation, split generation, label derivation, feature generation, model execution, cross-dataset evaluation, metric computation, and run documentation explicit parts of the research method.

Six design principles guide the methodology. First, *separation of concerns* keeps acquisition, signal preparation, benchmark construction, model execution, and result inspection as separate stages. This prevents dataset-specific parsing decisions from becoming entangled with model logic. Second, *declarative reproducibility* requires experimental choices to be written as configuration rather than hidden in notebooks or manual scripts. Third, *provenance preservation* requires the benchmark to retain enough information to trace a metric back to the source

recordings, split policy, label policy, model configuration, and evaluation definition that produced it. Fourth, *extensibility* treats datasets, annotation adapters, feature builders, and algorithms as replaceable components. Fifth, *cross-dataset evaluation by construction* treats the source training dataset and the target evaluation dataset as separate axes of every experiment, so that within-dataset and cross-dataset performance are produced by the same workflow rather than by ad-hoc additional studies. Sixth, *conservative supervision* favours unambiguous AF-vs-normal examples when constructing the first benchmark labels, even if that means excluding some rhythm segments from supervised training and primary evaluation.

This framing positions the thesis contribution as a reproducible benchmarking framework supported by demonstration detectors that exercise the framework, which is itself the contribution. Signal processing and machine learning algorithms remain central to the project, but they are compared as plugins inside a shared experimental workflow. The value of the framework is that the same methodological rules can be applied across classical RR-based detectors, handcrafted-feature models, and waveform-based models, and across every combination of training and evaluation dataset.

3.1.1 Summary of Key Design Decisions

Several methodological choices were fixed early in the design as defaults while deliberately being kept configurable, so that the framework expresses a defensible first benchmark without hard-coding it. Table 3.1 records these decisions, the default selected for the first implementation, the configurable alternatives, and the reason the default is reasonable. The remainder of this chapter develops each decision in detail.

Table 3.1: Key methodological design decisions, their defaults, configurable alternatives, and rationale.

Decision	**Default**	**Config options**	**Design rationale**
Prediction target	Window and recording level	-	Windows give the supervised unit; recordings give a clinical/workflow view.
Binary label mapping	AF vs NSR (conservative)	`af_vs_non_af`, `af_or_afl_vs_nsr`, `af_or_afl_vs_non_af`	Cleanest supervision for the first benchmark; broader mappings remain configurable.

Decision	Default	Config options	Design rationale
Window length / stride	10 s window, 5 s stride	Any positive window length and stride (stride $\leq$ window)	Enough RR context for rhythm-level AF evidence with overlapping coverage.
Target sampling rate	250 Hz	Any processed sample-rate folder	Adequate for rhythm analysis; reduces storage.
Lead usage	Single lead	Configurable preferred-lead priority list; single-lead policy only	Common denominator across heterogeneous datasets and lead names.
Train/val/test organisation	Per source dataset, configurable	Split ratios; stratify by `af_present`, `dataset`, `dataset_af_present`, or `none`	Avoids leakage and supports cross-dataset evaluation per experiment.
Decision threshold	Tuned on source validation split (default 0.5)	-	Reporting is honest when the operating point is chosen without seeing the target.
Recording score aggregation	Mean of window probabilities	`max`, `majority_vote`; recording truth-label policy `any` / `majority` / `fraction_threshold`	Reflects overall AF burden; max/majority remain configurable.
Feature/window engine	PySpark	-	Scales window generation over the processed parquet corpus.
Output storage	Object store artifacts plus relational catalog	Configurable buckets/prefixes; store types fixed	Durable, inspectable artifacts with a queryable metadata index.
Distribution model	Reusable package plus CLI	-	Lets others run and extend the comparison reproducibly.
First model set	Baselines first, documented model API	Any subset of supported models (DSP/RR, feature-ML, raw-waveform)	Establishes comparable references before adding richer detectors.

3.2 Dataset Curation and Selection

Benchmark construction begins before any model is trained. In the initial survey, approximately 50 ECG datasets were identified as potentially relevant to AF detection or rhythm-analysis research. These candidate datasets were screened according to practical and methodological criteria: whether they were accessible, whether they contained rhythm or event annotations relevant to AF, whether their technical format could be read reproducibly, whether their documentation was sufficient, and whether their population or acquisition protocol contributed useful diversity. The broader survey is listed in Appendix A.

These criteria are methodological rather than purely technical: a dataset is

useful only if it can support reproducible labels, defensible splits, auditable preprocessing, and practical execution within the first implementation scope.

During the design stage, several ways of documenting dataset-specific knowledge were considered. The chosen approach is to use dataset manifests: manually maintained, human-readable configuration files that serve both the pipeline and the researcher. Each manifest records the information needed before ingestion, including the dataset name, provider, download route such as a `download_url` or access instruction, citation or licence information, expected annotation type, adapter choice, and known quirks. The downloader module can then read the available manifests and start the dataset-fetching process without hard-coded dataset logic, while researchers can inspect the same files as a compact catalogue of which datasets are present and how they are expected to be handled. Technical properties that are encoded in the downloaded files themselves, such as sampling rate, channel names, duration, number of samples, and available annotation files, are inferred during processing rather than duplicated manually in the manifest.

3.3 Data Pipeline Design

The data pipeline is designed as the lower layer of the benchmark. Its role is to convert heterogeneous ECG resources into a common, auditable representation that can be consumed by later benchmark runs. At this stage the goal is to produce a standardised recording catalogue with waveform data, annotation information, and provenance.

3.3.1 Overall Architecture

The designed ingestion flow has three conceptual stages:

```
acquire source data -> standardise recordings -> register
                                                metadata
```

Figure 3.2 expands this sequence into the intended data movement and outputs of each stage.

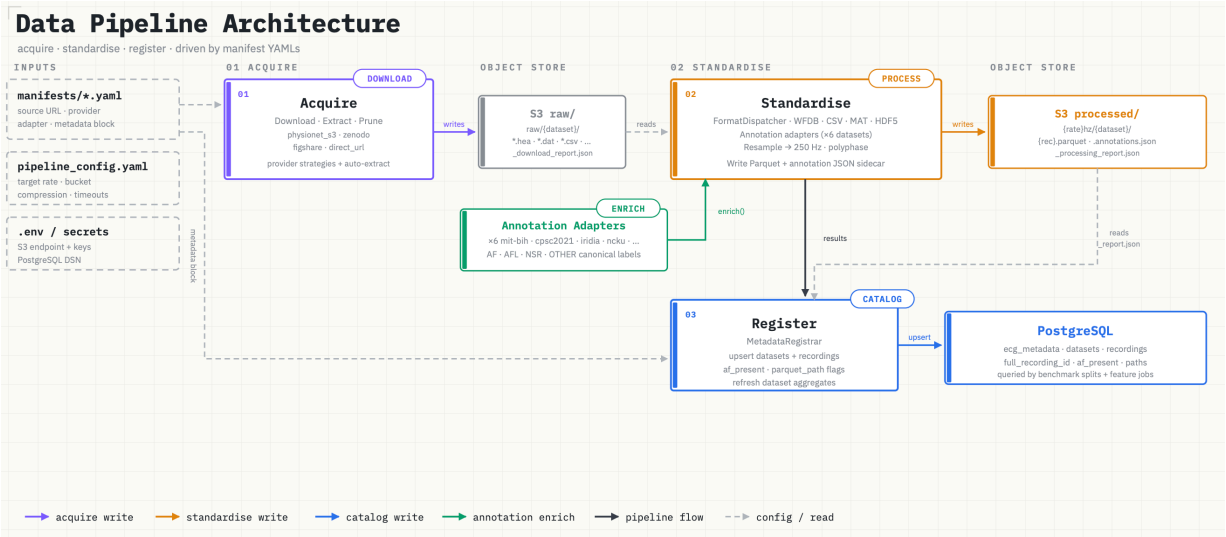


Figure 3.2: Data-pipeline design from curated manifests and raw downloads to standardised waveforms, annotation sidecars, and metadata registration.

The acquisition stage is responsible for obtaining the original files and preserving their source layout as far as possible. Provider-specific download differences should be isolated behind acquisition strategies so that adding a new hosting platform does not require changes to signal standardisation or benchmark evaluation. This separation also supports auditability: the processed corpus can always be related back to the source files used to create it.

The standardisation stage is responsible for discovering recordings, selecting readable ECG channels, normalising sampling rates, and separating waveform data from annotation data. Waveforms are represented in a consistent time-indexed form, while annotations are stored as sidecar information that can evolve independently from the waveform files. This design makes it possible to improve annotation adapters or label mappings without rewriting large waveform objects.

The metadata-registration stage creates a catalogue of available recordings in a relational database. The catalogue records dataset identity, recording identity, subject identity when available, sampling rate, duration, channel information, annotation availability, AF availability, and pointers to the standardised waveform and annotation objects. Later benchmark runs query this catalogue instead of scanning the storage layer for every experiment, which keeps splits and feature generation tied to registered metadata.

3.3.2 Sampling Rate Standardisation

The ECG datasets considered in this thesis do not share a single sampling frequency. Some recordings are acquired at relatively low rates suitable for long-term rhythm monitoring, while others are stored at higher diagnostic sampling rates. A common sampling rate is therefore needed so that feature extraction, window construction, and waveform-model input sizes are comparable across datasets. The default target rate selected for this work is 250 Hz.

The 250 Hz target is a practical compromise. It matches the native sampling rate of the MIT-BIH Atrial Fibrillation Database, a widely used AF reference dataset [63]. At this rate each sample represents 4 ms, which is adequate for rhythm-level AF analysis based on RR-interval irregularity, absence or instability of P waves, and episode-level rhythm labels. It also reduces storage and downstream input size compared with 500 Hz or 1000 Hz sources while preserving the signal content most relevant to AF detection.

Several rate-conversion strategies were considered before selecting the sampling-rate standardisation approach. Table 3.2 summarises the main advantages and limitations of each option.

Based on this comparison, the selected design is FIR-based polyphase resampling with anti-alias filtering. This option was chosen because it provides a controlled transformation between source and target sampling rates while remaining suitable for both downsampling and upsampling across heterogeneous ECG datasets.

For a source rate f_s and target rate f_t, the conversion can be represented as a rational factor L/M, where L is the upsampling factor and M is the downsampling factor after reduction by the greatest common divisor. The ECG signal is filtered and resampled along the time axis for each selected lead. The method supports both downsampling and upsampling, but the benchmark records the original sampling rate so that later analyses can distinguish native 250 Hz recordings from converted ones.

3.3.3 Annotation Normalisation

AF datasets vary in both waveform format and annotation semantics. Some datasets provide rhythm intervals, some provide event intervals, some provide beat

Table 3.2: Rate-conversion strategies considered for ECG sampling-rate standardisation.

Strategy	**Advantages**	**Limitations**
Simple decimation	Computationally cheap and easy to implement.	Does not provide anti-alias filtering, so high-frequency components can fold into the lower-frequency band during downsampling.
Linear or spline interpolation	Useful for estimating values between existing samples and for simple upsampling tasks.	Less appropriate as the main downsampling method because it does not by itself define a controlled anti-alias filtering step.
Fourier-based resampling	Can provide accurate rate conversion for some signals.	Assumes periodicity at signal boundaries and may introduce edge artefacts in long ECG recordings.
FIR-based polyphase resampling	Provides controlled rate conversion with anti-alias filtering and supports rational upsampling and downsampling.	More computationally involved than simple decimation or interpolation.

annotations, and others provide recording-level or dataset-specific labels. The pipeline is therefore designed to preserve source annotations and then derive benchmark labels through explicit adapter rules. The source annotation should not be overwritten by a single binary label during ingestion, because future experiments may need a different label policy.

The design uses a normalised annotation sidecar for each recording. The sidecar can contain rhythm intervals, event intervals, beat locations, source label strings, mapped rhythm categories, and adapter metadata. This approach preserves enough information to support multiple benchmark tasks: a conservative AF-vs-NSR task for the first comparison, broader rhythm classification in future work, or dataset-specific episode analysis when required. The dataset-adapter concept that enables this normalisation is explored as part of the data-pipeline implementation in Chapter 4, Section 4.1.

3.3.4 Processed Waveform Serialisation Format

Once a recording has been resampled and its annotations normalised, the standardised waveform must be written to a persistent format that will be read repeatedly by every downstream benchmark step. Choosing this format is a design decision with real consequences for storage efficiency, read throughput, query capability, and compatibility with the distributed processing layer. Because the pipeline ingests heterogeneous source files—WFDB header–data pairs (`.hea`/`.dat`), MATLAB matrices (`.mat`), HDF5 containers (`.h5`/`.hdf5`), plain comma-separated tables (`.csv`), and WFDB annotation streams (`.atr`/`.qrs`)—a single unified output format is essential. Without it, every downstream component would need format-specific reading logic, reintroducing the heterogeneity that the pipeline is intended to resolve.

Alternatives considered. Four candidate formats were evaluated before the design was fixed.

WFDB format. WFDB is the native representation of most PhysioNet corpora and the format from which a large share of the pipeline's source data is read. It is well understood in the ECG community and supported by WFDB tooling [64]. Its limitations as a processed output format are, however, significant. WFDB stores each recording as a binary signal file paired with a plain-text header, which means

reading a single column—for example, one lead—from a long recording requires loading the full binary stream. The format has no native concept of partitioning, predicate pushdown, or schema enforcement that would allow a distributed engine such as PySpark to parallelise access without custom reading wrappers. It is therefore well suited to raw archival but not to large-scale analytical queries over a growing corpus.

JSON. JSON is human-readable, self-describing, and trivially portable [65]. However, ECG waveforms are dense numerical time series: a single 30-minute recording at 250 Hz contains 450 000 floating-point samples per lead. Encoding these as text is between one and two orders of magnitude less space-efficient than binary representations. Parsing overhead is similarly large, and JSON provides no column-projection or row-filtering capability that could avoid loading data not needed by a query. JSON is retained as a format for run configuration, split manifests, and annotation sidecars—where human readability matters and data volume is small—but it is unsuitable for waveform serialisation.

Avro. Apache Avro is a row-oriented binary format with strong schema evolution guarantees, which makes it attractive for data interchange and message streaming [66]. For an analytical workload that repeatedly reads only a subset of columns from many recordings, however, row orientation is a disadvantage: accessing a single lead requires scanning every field of every row. Avro is therefore best matched to write-heavy streaming pipelines rather than the read-heavy, column-selective queries that window and feature generation perform.

HDF5. HDF5 is widely used in scientific computing and signal processing, and several ECG datasets in the survey store their recordings directly in HDF5 containers. HDF5 supports hierarchical organisation, arbitrary metadata attributes, and efficient storage of multi-dimensional arrays [67]. Its main limitations in a distributed benchmark context are file-level locking semantics and the absence of native partitioning primitives. Parallel reads from a single HDF5 file require careful coordination, and Spark has no first-class HDF5 connector; custom reader logic would need to be distributed to every executor node. For single-machine exploratory analysis HDF5 is effective, but the design targets a workflow that should scale to a cluster without code changes.

Selected format: Apache Parquet with Snappy compression. The design uses Apache Parquet as the serialisation format for processed waveforms, with Snappy selected as the compression codec.

Parquet is a columnar binary format originally designed for analytical workloads over large tabular datasets [68]. Its column-oriented physical layout means that reading only the signal column for one lead, or only the metadata columns needed to decide which recordings to process, does not require loading the remainder of each row. For the benchmark's dominant access pattern—reading one lead's waveform samples for a set of recording windows—columnar storage reduces unnecessary I/O materially compared with row-oriented alternatives. Parquet also enforces a typed schema, which provides an implicit consistency check: if a processed file were written with an unexpected column type, Spark or PyArrow would raise an error at read time rather than silently producing malformed data.

Most importantly for this framework, PySpark treats Parquet as a first-class format. Predicate pushdown allows Spark to skip row groups whose metadata does not satisfy a filter before any signal data is read. Partition pruning allows the corpus to be organised by dataset or recording identifier so that runs touching a subset of datasets avoid reading irrelevant files entirely [69]. These capabilities are available without changing the benchmark code when execution moves from a single machine to a cluster, which satisfies the design requirement that the pipeline should be horizontally scalable by configuration rather than by code modification.

Snappy was selected as the compression codec over the main alternatives—gzip, zstandard, and uncompressed storage—for a specific reason. A benchmark that iterates over the same processed corpus many times is read-dominated: the pipeline writes each processed file once but feature generation, window construction, and model evaluation read it many times. In this regime, decompression speed matters more than the compression ratio. Snappy is optimised for fast decompression at the cost of a modestly lower compression ratio compared with gzip or zstandard [70]. For the floating-point waveform data typical of ECG recordings, Snappy achieves a useful reduction in storage and network transfer while adding negligible latency to repeated reads. Snappy is also the default Parquet codec in many Spark deployment configurations, which avoids compatibility surprises when the processed corpus is read by external tools or shared with collaborators.

Annotation sidecars. Each recording's annotations are stored in a separate JSON sidecar alongside the waveform Parquet file. The sidecar carries rhythm intervals, event intervals, beat locations, source label strings, mapped rhythm categories, and adapter metadata. Keeping waveform and annotation data in separate files means that an improved annotation adapter or a revised label mapping can be applied and the sidecar replaced without rewriting the potentially large waveform object. It also means that annotation logic can be inspected and tested using standard text tooling before any waveform data is involved.

Summary of the format decision. After ingestion, the pipeline converts heterogeneous raw recordings into a corpus of Parquet waveform files and JSON annotation sidecars. Source format diversity is handled entirely within the acquisition and standardisation stages; every downstream benchmark component reads the same Parquet schema and the same annotation structure regardless of which dataset the recording originates from. The choice of Parquet with Snappy compression balances storage efficiency, read performance, schema enforcement, and first-class Spark integration in a way that none of the alternatives provides across all four criteria simultaneously.

3.4 Benchmark Package Design

The benchmark layer is designed to sit above the processed corpus. It assumes that standardised waveform objects, annotation sidecars, and recording metadata are available, and it defines how those resources become reproducible experiments. Its main responsibilities are configuration validation, split creation, window and feature generation, label derivation, model execution, cross-dataset evaluation, metric calculation, timing measurement, and result documentation. To make these responsibilities reusable beyond a single study, the benchmark is designed as a software package with a command-line interface, so that other researchers can run, reproduce, and extend the comparison without rewriting the workflow. Figure 3.3 summarises this package-level architecture and shows how configuration, data access, experiment orchestration, model plugins, evaluation, and stored artefacts fit together.

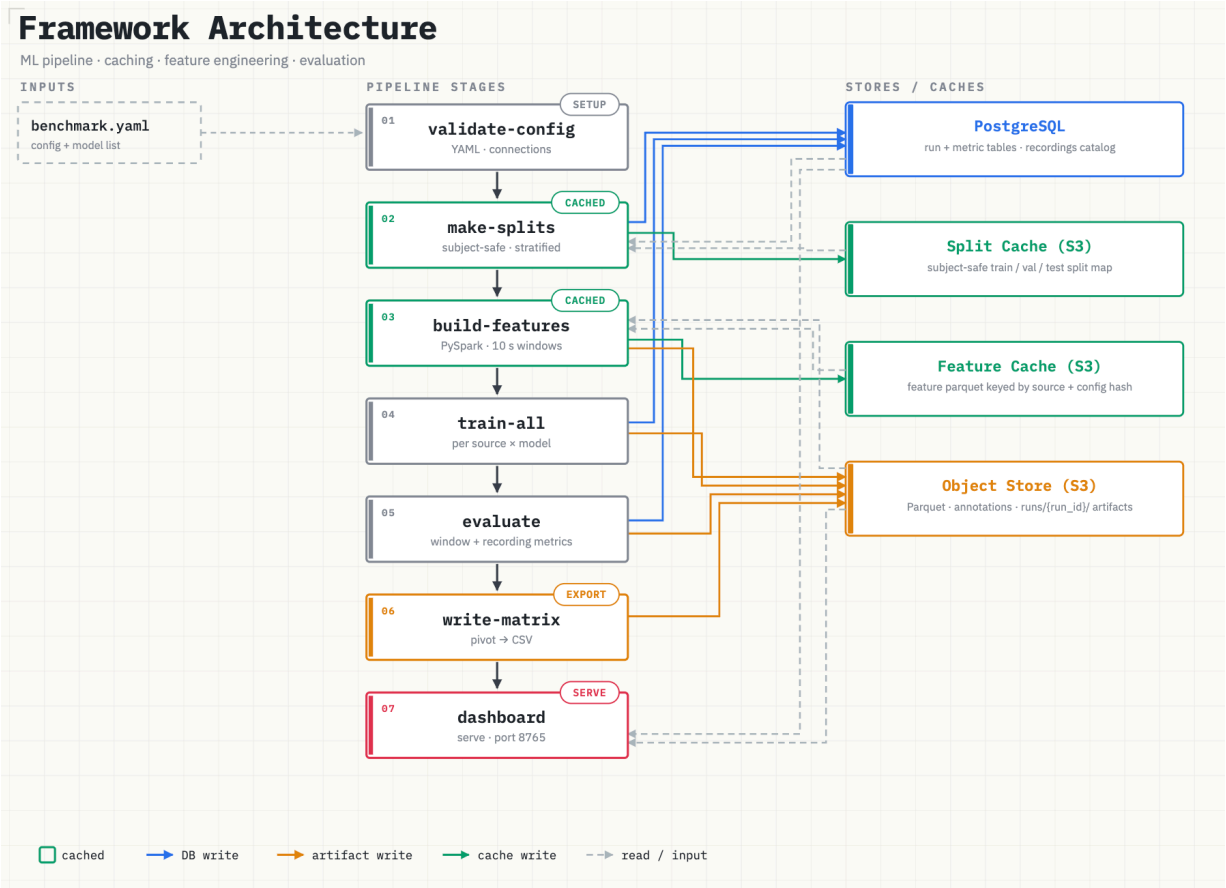


Figure 3.3: Benchmark package architecture design, showing the main workflow from declarative configuration and processed ECG inputs through split generation, windowing, model execution, evaluation, and persisted artefacts.

3.4.1 Declarative Experiment Specification

Each experiment is defined by a declarative configuration. The configuration describes selected source and target datasets, target sampling rate, window length, stride, split policy, label policy, lead policy, feature representation, model set, model parameters, metrics, aggregation policies, and output-location choices. Defaults may be provided by the framework, but a resolved configuration should be saved with every run so that implicit values do not disappear.

The configuration is also the basis for run identity. A deterministic hash of the resolved configuration links a result to the settings that produced it. This provides a compact guard against accidentally comparing results from different window sizes, label policies, split seeds, lead policies, or model parameters.

3.4.2 Cross-Dataset Experiment Design

A defining methodological feature of the framework is that the source training dataset and the target evaluation dataset are treated as independent axes of every experiment. The configuration declares a set of source datasets and a set of target datasets. For each configured model, one independent model instance is trained on each source dataset, and each trained instance is then evaluated on every target dataset. Source and target datasets are not merged into a single pooled corpus; keeping them separate is what makes both within-dataset performance, where source equals target, and cross-dataset generalisation, where source differs from target, fall out of the same run.

The outcome of a run is therefore naturally a model$\times$source$\times$target structure rather than a single score table. Figure 3.4 illustrates the resulting evaluation matrix for one model: diagonal cells report within-dataset performance, and off-diagonal cells report transfer to an unseen dataset.

This design directly shapes how train, validation, and test splits are organised, which is described next.

3.4.3 Split Design

Splits are generated per dataset and composed into a run-level split manifest. A model is fitted on the training partition of its source dataset. Its validation

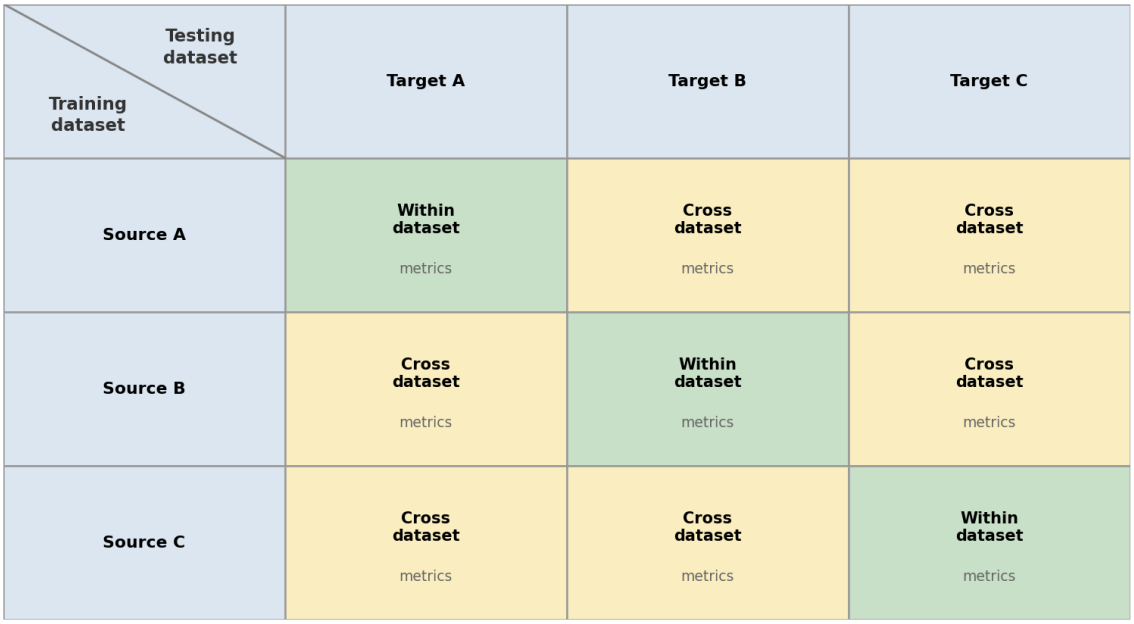


Figure 3.4: Cross-dataset evaluation matrix for one model. Each trained source instance is evaluated on every target dataset; diagonal cells are within-dataset results and off-diagonal cells are cross-dataset transfer results.

partition is reserved for model selection, hyperparameter selection, or threshold selection when such tuning is enabled. Reported metrics come from the test partition of each target dataset. Organising splits per dataset, rather than as one global split, keeps the cross-dataset axis meaningful and configurable at the experiment level.

Split generation is deterministic for a given configuration and random seed. When subject identifiers are available, all recordings from the same subject remain in the same split to reduce the risk of patient leakage. When only recording identifiers are available, the framework falls back to a recording-level grouping policy and documents that limitation. Stratification can be applied to AF presence, dataset identity, or both, depending on the experimental objective; AF presence is the default because it keeps AF-positive and AF-negative groups balanced across the three partitions. The split manifest is a benchmark artefact, not a permanent mutation of the ingested dataset.

3.4.4 Windowing and Label Policy

The primary supervised unit of the benchmark is an ECG window. A window has a dataset identifier, recording identifier, start time, end time, the selected lead, input representation, and derived label. Window length and stride are config-

urable because AF detectors may target different temporal scales: short windows are useful for local rhythm evidence, while longer windows provide more RR-interval context. The first benchmark uses a default window length of 10 s with a 5 s stride, which provides enough beats for RR-interval irregularity evidence while overlapping windows give denser coverage of each recording.

The default binary label policy is conservative AF-vs-NSR. A window is labelled as AF only if it is fully contained inside an AF interval. It is labelled as NSR only if it is fully contained inside a normal sinus rhythm interval. Windows that cross episode boundaries, or that overlap rhythm labels such as atrial flutter, other rhythm, or unknown/unmapped source labels, are excluded from the primary supervised task. Figure 3.5 illustrates this rule.

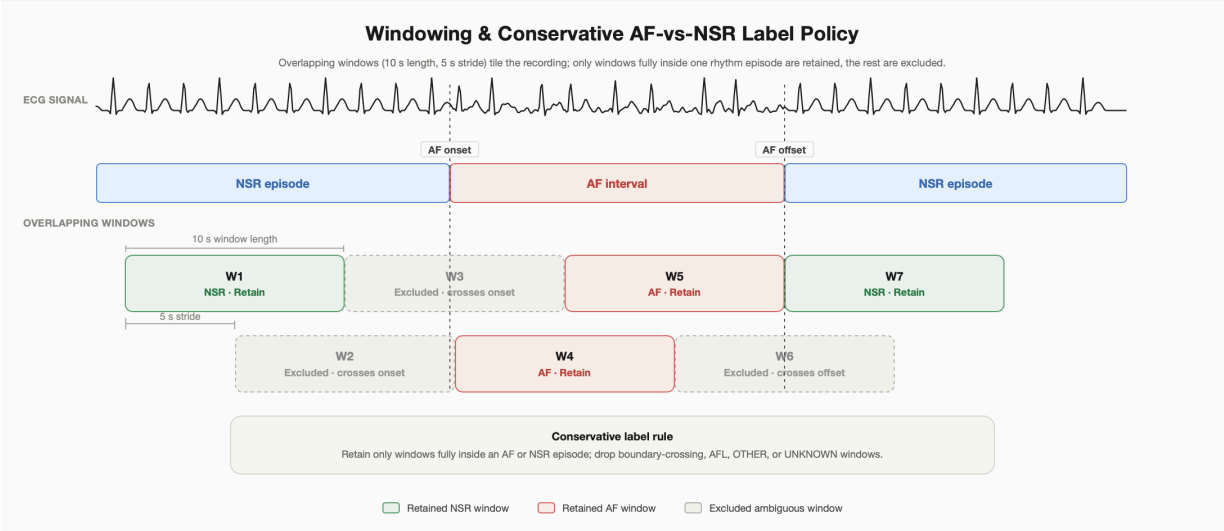


Figure 3.5: Conservative AF-vs-NSR window labelling: only windows fully contained in a single accepted rhythm interval are retained for the primary supervised benchmark.

This policy sacrifices some data coverage in exchange for clearer supervision. The rejected windows are not necessarily clinically unimportant; they are excluded because their labels are ambiguous for the first binary benchmark. The label mapping is configurable: alternative policies can treat all non-AF rhythms as the negative class, or group atrial flutter with AF, when an experiment requires it. The primary comparison deliberately avoids mixing AF, atrial flutter, transition regions, and uncertain labels into the same training target.

3.4.5 Single-Lead Input Policy

The datasets considered here do not share a common set of named leads, and several are single-lead or use different channel naming conventions. To keep windows comparable across datasets, the framework standardises on a single-lead input policy: for each recording, one ECG lead is selected from a configurable priority list, with a deterministic fallback when no preferred lead is present. Training and prediction therefore operate on one lead, which is the practical common denominator across heterogeneous sources and is also relevant for wearable and single-lead monitoring applications. Multi-lead extensions are left to future work and are not required by the AF rhythm evidence targeted in the first benchmark.

3.4.6 Feature and Window Generation

Window generation and feature extraction are designed to run as a scalable step over the processed Parquet corpus. The feature builder reads processed waveforms and annotation sidecars, enumerates candidate windows per recording, derives labels with the configured label policy, and writes two complementary representations:

- a *tabular* window table containing RR-interval summaries and simple signal statistics, consumed by classical and feature-based machine-learning models; and
- a *waveform* window manifest that records, for each window, a pointer to the processed signal, the selected lead, and the start/end samples, so that waveform models can load raw segments lazily rather than materialising every sample up front.

Separating the two representations lets RR-based detectors, handcrafted-feature models, and raw-waveform models share the same windows, labels, and splits while consuming the input form each family needs.

3.4.7 Input Representations and Model Contract

The benchmark is designed to support more than one algorithm family. Classical signal-processing models and many machine-learning baselines use tabular

features derived from beats, RR intervals, or signal summary statistics. Waveform models instead require access to raw or lightly processed ECG segments. The framework therefore distinguishes between tabular-window inputs and waveform-window inputs while keeping the downstream scoring interface consistent.

All algorithms are treated as benchmark plugins. A plugin must declare its identifier, input kind, required feature columns or waveform requirements, trainability, and configurable parameters. It must also provide a way to fit model state when needed, produce AF probabilities for benchmark windows, and persist or reload its learned artefacts. Table 3.3 summarises the design contract.

Table 3.3: Design contract for benchmark model plugins.

Capability	**Requirement**
Metadata declaration	The model states its identifier, input representation, required fields, trainability, and configurable parameters.
Training behaviour	Trainable models learn from training windows and may use validation windows for tuning; deterministic models document that no learned parameters are required.
Probability scoring	Every model returns an AF probability for each evaluated window so that common threshold-dependent and threshold-independent metrics can be computed.
Persistence	Learned state and configuration-dependent artefacts can be saved and restored as part of the run provenance.
Interchangeability	Adding a new model should not require changes to split generation, feature generation, label derivation, metric calculation, or result storage.

Because every model exposes the same probability-scoring interface, the framework can compare detector families that would normally be evaluated under incompatible protocols: rule-based RR detectors, handcrafted-feature classifiers, and raw-waveform neural networks are all scored by identical benchmark logic

over identical windows and splits.

3.4.8 Reuse and Caching

Cross-dataset experiments repeat a lot of work: changing only the model set, or adding one new dataset, should not force every split and every window to be regenerated. The design therefore separates dataset-level preparation from run-level execution and caches the expensive, reusable products of preparation. Split assignments and generated feature or waveform-window outputs are reusable when the inputs that affect them, such as dataset membership, sampling rate, window length, stride, label policy, lead policy, split ratios, stratification, and random seed, remain unchanged. Model identifiers and metric lists deliberately do not affect these reusable preparation products.

This has two methodological consequences. First, a model-only or metric-only change can reuse compatible partitions and windows, which makes iterative comparison cheaper and keeps repeated runs consistent. Second, adding a new dataset to a later experiment can reuse preparation products for the existing datasets, so the cross-dataset matrix can grow incrementally. Each run should still record exactly which split and feature manifests were used, preserving provenance even when cached preparation is reused.

3.4.9 Artefact and Provenance Requirements

A benchmark result is useful only if it remains inspectable after the run has finished. The design therefore treats durable artefacts as part of the scientific output, and it separates two complementary storage responsibilities: durable artefact storage for files and a relational catalogue for queryable metadata about recordings, runs, models, and metrics. At minimum, a run should preserve the resolved configuration, split manifest, generated window and feature manifests, model artefacts, window-level predictions, recording-level aggregated predictions, metric tables, timing records, and a human-readable run summary. Chapter 4 describes the concrete storage layout used for these artefacts.

This requirement avoids making notebooks or local scratch folders the authoritative result store. A notebook may visualise a run, but the reproducible state of the experiment should live in the run artefacts. This also allows command-line

tools, dashboards, and later analyses to read the same outputs without creating separate result systems.

3.5 Evaluation Methodology

Evaluation is organised around two levels of interpretation. Window-level evaluation measures whether a detector assigns correct AF probabilities to labelled ECG windows. Recording-level evaluation aggregates those probabilities to ask how the detector behaves when its window decisions are summarised over a full recording. Window-level metrics are the primary benchmark output because they are closest to the labels used for supervised comparison; recording-level metrics provide a secondary clinical and workflow view. Both levels are computed for every source×target pair, so classification quality is always reported with its training and evaluation context.

3.5.1 Window-Level Metrics

For binary AF-vs-NSR windows, the benchmark reports threshold-dependent metrics derived from true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN). The core threshold-dependent quantities are:

$$\begin{aligned} \text{Sensitivity} &= \frac{TP}{TP+FN}, \qquad \text{Specificity} = \frac{TN}{TN+FP}, \\ \text{Precision} &= \frac{TP}{TP+FP}, \qquad F1 = \frac{2 \cdot \text{Precision} \cdot \text{Sensitivity}}{\text{Precision} + \text{Sensitivity}}. \end{aligned} \tag{3.1}$$

Sensitivity and recall are identical quantities; the framework exposes both names so that users can select whichever is conventional in their reporting context. Two further threshold-dependent metrics are available as non-default opt-in quantities. The negative predictive value (NPV) captures the fraction of negative predictions that are correct:

$$\text{NPV} = \frac{TN}{TN+FN}. \tag{3.2}$$

Balanced accuracy summarises performance across both classes indepen-

dently of class prevalence by averaging sensitivity and specificity:

$$\text{Balanced Accuracy} = \frac{\text{Sensitivity} + \text{Specificity}}{2}. \tag{3.3}$$

The operating threshold used for all of the above threshold-dependent metrics is a methodological choice in itself. Because the framework reports cross-dataset results, the design selects the decision threshold on the *source* validation split and then applies it unchanged to every target test set, with a default operating point of 0.5. Choosing the threshold without access to the target keeps cross-dataset reporting honest and prevents specificity or sensitivity from being silently optimised on the evaluation data.

Because all benchmark models produce AF probabilities, threshold-independent metrics are also included. The area under the receiver operating characteristic curve (AUROC) measures ranking quality across all classification thresholds, while average precision summarises the precision–recall curve and is especially informative when the AF and NSR classes are imbalanced. To make label composition visible beside the scores, each result row also reports the number of evaluated windows and the positive and negative counts. Accuracy may be reported, but it is not sufficient as the main metric for imbalanced rhythm detection tasks, and metrics that are undefined for a given evaluation (for example AUROC when a target test set contains a single class) are recorded explicitly as undefined rather than as a misleading value.

The nine metrics reported by default are `n`, `positive`, `negative`, `auroc`, `average_precision`, `accuracy`, `f1`, `sensitivity`, and `specificity`. The four additional opt-in metrics are `precision`, `recall`, `npv`, and `balanced_accuracy`; these can be selected via the `metrics` list in the benchmark configuration.

3.5.2 Recording-Level Aggregation

Long-term ECG monitoring is often interpreted at recording or episode level rather than as isolated windows. The framework therefore defines secondary recording-level outputs by aggregating the AF probabilities predicted for all evaluated windows in a recording. The probability aggregation policy can be selected according to the experimental question. A mean policy, used as the default, reflects overall AF burden across the evaluated windows. A maximum

policy reflects whether any window strongly suggests AF. A majority-vote policy reflects the dominant predicted rhythm after thresholding individual windows.

The recording-level truth label must also be derived from window labels. With an `any` policy, a recording is positive if at least one evaluated window is labelled as AF. With `majority`, a recording is positive if at least half of its evaluated windows are AF. With `fraction_threshold`, a recording is positive when the AF-window fraction reaches a selected threshold. The default design uses a fraction threshold of 0.5 so that the recording label represents the dominant rhythm over the evaluated windows. Experiments focused on detecting any occurrence of AF can use the `any` policy instead.

This design keeps window-level supervision and recording-level interpretation separate. It also makes the aggregation rule visible, which is important because different aggregation policies answer different clinical and computational questions.

3.5.3 Computational Cost

In addition to classification quality, the benchmark treats computational cost as a first-class outcome. This is methodologically important because an AF detector that achieves high AUROC but requires long training cycles or slow prediction may be unsuitable for iterative research, wearable deployment, or large-scale retrospective analysis.

The design separates three timing concepts. Training time measures model fitting and persistence of any learned state. Inference time measures the model's probability scoring on target windows. Evaluation time measures the broader envelope around inference, including loading inputs, aggregating recording outputs, computing metrics, and writing artefacts. Separating these durations prevents model latency from being confused with surrounding data-management work.

Cost is reported together with the number of processed windows, the model identifier, the source training dataset, the target evaluation dataset, and the input representation. This allows later analysis to compute per-window latency and compare detector families by both quality and computational burden. Rule-based detectors that do not learn parameters are recorded with zero benchmark training cost while preserving their measured fit or setup time separately, so that zero training cost is not interpreted as missing timing data.

3.5.4 Reproducibility and Extensibility Criteria

The framework is evaluated by both the classification results produced during demonstration runs and whether it supports reproducible and extensible experimentation. Reproducibility means that the same dataset selection, preprocessing settings, split policy, label policy, lead policy, model configuration, and metric definitions can be used to repeat an experiment and inspect how its results were produced. Extensibility means that new datasets, annotation adapters, label policies, feature builders, or model implementations can be added without redesigning the complete benchmark.

The practical criteria used to assess the design are therefore: configurations are validated before execution; run identifiers are linked to resolved settings; train, validation, and test splits are deterministic and reusable; source annotations remain traceable; source and target datasets are evaluated as separate axes; models return predictions in a shared probability format; decision thresholds are chosen without access to target test labels; metrics are computed by common benchmark logic; computational cost is recorded alongside classification quality; and durable artefacts remain available for inspection after the run.

3.6 Chapter Summary

This chapter defined the benchmark as a methodology before describing any implementation. The design standardises heterogeneous ECG recordings to a common 250 Hz single-lead representation, preserves source annotations, defers experimental splits, generates fixed-width windows, constructs conservative AF-vs-NSR labels, and treats the training source and evaluation target as separate axes so that within-dataset and cross-dataset performance are produced by the same workflow. Detectors from different families are compared through a shared probability-scoring contract behind a reusable package and command-line interface; results are evaluated at both window and recording level alongside computational cost; reusable preparation can be cached to make the cross-dataset matrix cheaper to grow; and durable artefacts are stored for provenance. Chapter 4 describes how these design choices were implemented in the current software artefact.

Chapter 4

Framework Implementation and Demonstration

This chapter describes how the design from Chapter 3 was implemented in the current software artefact. Whereas Chapter 3 defines the benchmark requirements and methodological choices, this chapter describes the concrete datasets, services, packages, commands, artefact layout, and demonstration workflow used to realise those choices.

The implementation follows the same two-layer structure as the design. The first layer is the `ecg-data-pipeline` package that downloads source ECG files, standardises recordings to a common Parquet representation, and registers metadata in a relational catalogue. The second layer is the `ecg_benchmark` package that creates hash-traceable benchmark runs, generates Spark feature files, trains registered model plugins per source dataset, evaluates predictions across target datasets, stores durable artefacts, and exposes the workflow through a command-line interface and a lightweight local dashboard. Both packages are delivered as installable Python projects with `pyproject.toml` declarations. A Docker Compose stack providing PostgreSQL, MinIO, pgAdmin, and optionally Airflow brings up the complete runtime environment from a single command; the infrastructure is described in Section 4.6.

4.1 Data Pipeline Implementation

The implemented data pipeline realises the three-stage flow designed in Section 3.3.1: download and extract raw files, read and standardise ECG recordings, and register metadata. The implementation is manifest-driven. Each dataset manifest stores curated source information and adapter choices, while technical signal properties are inferred from the recordings at processing time.

4.1.1 Implemented Corpus

The current corpus is deliberately limited to a small set of datasets that can exercise the pipeline end-to-end. It should not be interpreted as the complete set of datasets identified in the survey. The implemented datasets are shown in Table 4.1; additional datasets from Appendix A remain future extension targets.

Table 4.1: Implemented datasets used to test source-provider access and file-parsing capabilities.

Dataset	**Source**	**Primary file types**
CPSC 2021 Paroxysmal AF Event Detection Dataset	PhysioNet	WFDB files: `.hea`, `.dat`, `.qrs`
MIT-BIH Atrial Fibrillation Database	PhysioNet	WFDB files: `.hea`, `.dat`, `.atr`, `.qrs`
IRIDIA-AF Database	Zenodo	HDF5 files: `.h5`; metadata and annotation `.csv` files
NCKU CBIC ECG Database	Figshare	CSV files

This conservative implementation boundary is important for the thesis claims. The framework supports extension to additional databases, but the current empirical demonstration is limited to datasets for which ingestion, annotation handling, and benchmark registration have been implemented.

4.1.2 Download and Extract

The downloader selects and executes a provider-specific strategy based on the `provider` field declared in each dataset manifest. When a provider is not specified explicitly, the pipeline infers it from the download URL through the

`infer_provider_from_url()` helper, so that manifests can use a bare URL without redundantly naming its host. Four strategies are implemented.

PhysioNet S3 strategy. PhysioNet public datasets are accessed via unsigned `boto3` requests against the `physionet-open` bucket on Amazon S3. The strategy lists the remote prefix corresponding to the requested database version, filters to the files declared in the manifest, downloads each file, and uploads it to the local MinIO instance under `raw/{dataset_name}/` while preserving the source directory layout. Using the S3 API rather than HTTP avoids reconstructing file trees from HTML index pages and provides checksum validation at the transport layer.

Zenodo strategy. Zenodo-hosted datasets are accessed through the Zenodo REST API. The strategy resolves the record identifier to obtain the file manifest, downloads each file via HTTPS, and optionally extracts archives before handing the resulting file tree to the processor. This strategy is used for the IRIDIA-AF Database.

Figshare strategy. Figshare-hosted datasets are accessed through the Figshare API, which returns a list of downloadable file objects for a given article identifier. The strategy iterates the file list, downloads each object, and unpacks archives as needed. This strategy is used for the NCKU CBIC ECG Database.

Direct URL strategy. A fallback HTTP strategy uses `requests` to download from an explicit URL. It handles redirect chains and optional archive extraction, and is registered for any provider that does not match the three named strategies above.

All four strategies share a common idempotency mechanism. Before issuing any network request the downloader calls `is_already_downloaded()`, which checks for any key under the raw prefix (excluding the download report file). If matching keys are found and `force_redownload` is not set, the download stage is skipped entirely. This prevents repeated pipeline runs from re-fetching large archives when only downstream processing or registration steps have changed.

After extraction, the pipeline runs `prune_raw_prefix()` to remove files not declared in the manifest and writes `_download_report.json` to the raw pre-

fix. The report records the set of files downloaded, their sizes, checksums, and timestamps. Together with the idempotency check and the original source layout preserved under `s3://ecg-pipeline/raw/dataset_name/`, this makes the downloaded corpus auditable against the processed files without re-fetching any data.

Provider-specific logic is kept outside the signal-processing code. Adding a new source therefore means adding or configuring a download strategy rather than changing the ECG readers, resampling logic, or benchmark evaluator.

4.1.3 Read, Parse, Resample, and Write

Format dispatch. The processing stage discovers recordings and dispatches them to format-specific readers through a `FormatDispatcher`. The dispatcher receives a list of reader instances and a file tree for the dataset; it matches each group of files to the appropriate reader by extension and passes control. The four implemented readers are:

- **WFDBReader** — handles PhysioNet waveform and annotation pairs (`.hea`, `.dat`, `.atr`, `.qrs`) using the `wfdb` library. The reader groups header and associated data files before reading, so that multi-file recordings are treated as a single unit.
- **MATReader** — handles MATLAB `.mat` containers via `scipy.io.loadmat`, with heuristic selection of the candidate array most likely to contain ECG signal and associated rate or channel metadata.
- **HDF5Reader** — handles `.h5` and `.hdf5` files via `h5py`, scoring candidate arrays by shape and dtype to select the ECG signal.
- **CSVReader** — handles plain tabular ECG files via `pandas.read_csv`. The reader infers the sampling rate from a time column or a header field and falls back to a configured default when neither is present.

Signal normalisation. After reading, every reader normalises the signal array to shape `[samples, channels]` and millivolt units via `normalize_signal_matrix()`

in `readers/_common.py`. This shared contract ensures that all downstream processing, including resampling, Parquet writing, and feature generation, can treat multi-lead and single-lead recordings interchangeably.

Resampling. All recordings are resampled to the configured target rate, 250 Hz in the thesis experiments, using polyphase filter resampling through `scipy.signal.resample_poly(signal, up, down)`. For a source sampling rate f_{src} and target sampling rate f_{tgt}, the implementation first reduces the conversion ratio to lowest terms: $g = \gcd(f_{\text{src}}, f_{\text{tgt}})$, $up = f_{\text{tgt}}/g$, and $down = f_{\text{src}}/g$. Using the reduced rational ratio avoids unnecessarily large intermediate arrays while preserving the exact source-to-target timing relationship. The polyphase filter applies the required low-pass anti-aliasing step before decimation, so the retained band is limited by the smaller Nyquist frequency, $\min(f_{\text{src}}, f_{\text{tgt}})/2$, rather than by an arbitrary dataset-specific choice.

Annotation timing is transformed at the same time as the waveform. Beat positions, rhythm episode sample bounds, and delineation markers are stored by source readers as integer sample indices at the original rate. The `adjust_annotations()` helper converts each index to the target sampling grid using:

$$i_{\text{new}} = \text{round}\left(i_{\text{old}} \frac{f_{\text{tgt}}}{f_{\text{src}}} \right). \tag{4.1}$$

At 250 Hz, rounding to the nearest target sample introduces a worst-case timing error of half a sample, $0.5/250 = 0.002$ s, or 2 ms. This error is small relative to the 10 s benchmark windows, the 5 s window stride, and the duration of rhythm episodes, so resampling preserves the alignment needed for window labelling and later metric computation.

Parquet output and schema metadata. The standardised waveform is written as a Parquet file using `pyarrow` with Snappy compression, as chosen in the design (Section 3.3.4). The columnar schema contains `sample_index` plus one column per ECG lead. Lead columns are named `channel_{i}_{name}`, encoding both the zero-based channel index and the original lead name. This naming scheme is what allows `select_single_lead()` to match preferred leads by name across datasets with different channel naming conventions.

In addition to the signal columns, the Parquet file carries PyArrow schema-level metadata encoding `recording_id`, `dataset_name`, `sampling_rate`, `original_sampling_rate`, `num_channels`, `channel_names` (as a JSON array), `duration_seconds`, `units`, `source_format`, `original_filename`, and `slice_index`. These embedded fields make every processed file self-describing: a Spark job or an interactive reader can recover the full recording context without consulting PostgreSQL.

Processed files are stored under `s3://ecg-pipeline/processed/250hz/dataset_name/`.

Annotation sidecar. Annotation information is written as a JSON sidecar adjacent to the Parquet file. The sidecar carries `recording_id`, `slice_index`, `annotation_source` (the adapter identifier that produced the annotation), `target_sampling_rate`, `original_sampling_rate`, `beat_annotations` (sample positions and labels), `rhythm_episodes` (start and end sample, rhythm category, and source label string), `delineation` (when available from the source dataset), and `labels` (adapter and mapped labels). The `annotation_source` field records which adapter was applied, providing an audit trail for any label mapping decision. Only fields that the source dataset provides and the adapter can extract are populated; absent fields are represented as `null` or an empty list rather than omitted, so that consumers can distinguish between *not present* and *not checked*.

The waveform Parquet output and the annotation sidecar are written to separate objects. This makes large waveform files stable while allowing annotation adapters or label mappings to be corrected without rewriting every sample-level signal file.

4.1.4 Metadata Registration

The registration stage writes structured metadata to a PostgreSQL catalogue managed by the `MetadataRegistrar` class. On startup, `ensure_schema()` executes `registrar/schema.sql`, which creates three tables if they do not already exist.

The `datasets` table stores one row per dataset: name, description, provider, licence, annotation types, and aggregate counts (total recordings, AF-positive

recordings, recording count with rhythm annotations). The `recordings` table stores one row per processed recording, keyed on `full_recording_id`. Recording rows carry dataset name, subject identifier (when available), current and original sampling rates, duration, channel names, number of samples, annotation availability flags, `af_present`, waveform Parquet path, and annotation sidecar path. The `resampled_recordings` table captures every rate variant produced for the same source recording, so that the benchmark can discover which target rates are available without listing the object store.

Registration uses `INSERT …ON CONFLICT DO UPDATE` semantics keyed on `full_recording_id`. Re-running the pipeline over an already-registered dataset updates existing rows in place rather than duplicating them, preserving idempotency end-to-end.

The `af_present` flag is derived in `processor.py` after annotation adapter execution. When rhythm episode intervals are available, the flag is set if any episode carries a mapped AF label. When no rhythm annotation is present, the adapter applies a label heuristic based on the recording-level diagnostic labels extracted by the adapter. This derivation is logged so that the flag can be traced to its source during downstream audits.

After bulk registration, `_refresh_dataset_aggregates()` recomputes recording counts and annotation-availability flags on the `datasets` row. This keeps the dataset-level summary consistent with the individual recording catalogue without requiring a separate maintenance query.

This metadata catalogue is the bridge between ingestion and benchmarking: the benchmark package can query available recordings and their properties without listing every object in storage.

4.2 Benchmark Package Implementation

The benchmark package is implemented as a separate layer above the data pipeline. It does not download raw datasets itself; it assumes that processed waveform Parquet files, annotation sidecars, and metadata records already exist. Its role is to turn those processed resources into executable, reproducible experiments.

4.2.1 Configuration Validation and Run Identity

Experiments are defined by YAML configuration files. The configuration is structured into a `BenchmarkConfig` dataclass with sub-sections for `DataConfig` (datasets, windowing, sampling rate, lead and label policies), `SplitConfig` (ratios, stratification, random seed), `StorageConfig` (S3 bucket, artifact and cache prefixes), `DatabaseConfig` (PostgreSQL environment variables), `SparkConfig` (master URL and executor settings), `ReuseConfig` (whether to reuse cached splits and features), and a list of `ModelConfig` entries. A compact baseline configuration is documented in Appendix B.1; the main text focuses on how the framework validates and uses that configuration.

Before execution, the framework resolves defaults and validates required fields. The default values applied when keys are absent are: 10 s window length, 5 s stride, 250 Hz target rate, 70/15/15 train/validation/test split ratios, stratification by `af_present`, and a decision threshold of 0.5. The `random_seed` field has no default and is required; validation raises a `ConfigError` with a user-facing message if it is absent. Additional checks verify that all named `model_id` values exist in the model registry, that dataset names resolve to processed data in the catalogue, and—when the `--check-services` flag is passed to `validate-config`—that the configured PostgreSQL and MinIO endpoints are reachable.

The configuration hash is computed as the SHA-256 of the canonical JSON serialisation of the fully resolved `BenchmarkConfig` dataclass:

```
sha256(json.dumps(asdict(config), sort_keys=True, separators=(",", ":")))
```

The canonical form uses sorted keys and compact separators so that the hash is independent of key-insertion order or formatting choices. The resulting 64-character hex digest is stored in three places: as the full `config_hash` column in the `benchmark_runs` PostgreSQL table, as the `config-hash` object metadata field on the S3 `resolved_config.json` artefact, and as the `config_hash` field inside `split_manifest.json`. Each location serves a different audit path: the database is queryable; the S3 object is self-describing; and the split manifest links a specific split to the configuration that generated it.

The run identifier is assembled from the UTC timestamp, the sanitised experiment name, and the first 12 characters of the configuration hash:

```
{YYYYMMDDTHHMMSSZ}-{experiment_name}-{config_hash[:12]}
```

A concrete example is `20260611T142024Z-showcase-smoke-5ee8640bec12`. The 12-character hash prefix is long enough to distinguish runs in practice while remaining readable in file paths and log output; the full hash is always available from the stored artefacts when exact identity is needed.

4.2.2 Command-Line Workflow

The command-line interface exposes the benchmark through the `ecg-bench` program, implemented with Python's `argparse` module and registered via the `pyproject.toml` entry-point `ecg-bench = "ecg_benchmark.cli:main"`. The implemented commands are:

- `validate-config`: resolves defaults, validates YAML schema and constraints (including model registry membership and presence of `random_seed`), optionally checks environment variables and service connectivity, and prints the configuration hash.
- `make-splits`: creates deterministic train, validation, and test assignments and persists the split manifest as a run artefact in S3 and in the `benchmark_splits` PostgreSQL table.
- `build-features`: builds window-level tabular features and waveform-window manifests for the configured datasets using PySpark.
- `train`: trains one configured model for one configured source dataset and writes the model artefact. The command accepts explicit `--model-id` and `--source-dataset` arguments; the `run` command loops over all combinations automatically.
- `evaluate`: loads trained source-specific models, scores target datasets, aggregates predictions to recording level, computes metrics, and writes prediction and timing artefacts.
- `matrix`: reads stored metrics for a completed run and writes wide benchmark matrices for comparison.
- `run`: executes the full workflow: validation, split generation, feature generation, training for all model $\times$ source combinations, evaluation of all trained

instances across all target datasets, matrix creation, and summary generation.

- `dashboard`: serves a local web interface for browsing runs, inspecting metrics, and generating configurations.

The CLI uses `argparse` exit codes to support shell scripting and CI integration: `ConfigError` conditions exit with code 2, and `RuntimeError` conditions exit with code 3, so that a calling script can distinguish a configuration mistake from a runtime failure without parsing output text.

The commands can be executed one at a time for inspection and debugging, or combined through `run` for a complete benchmark execution. Running them individually is useful when, for example, a split manifest already exists from a previous run and should be reused, or when a single model needs to be retrained after a hyperparameter change without repeating the full workflow.

4.2.3 Deterministic Splits and Conservative Labels

Split algorithm. The split implementation in `splits.py` follows a five-step deterministic procedure for each dataset:

1. Recordings are sorted by `(dataset_name, full_recording_id)` so that the input order is canonical and independent of catalogue query ordering.

2. Each recording is assigned to a group: recordings with a known subject identifier form group `subject:{dataset}:{subject_id}`, while recordings without a subject identifier form group `recording:{dataset}:{full_recording_id}`. This subject-safe grouping ensures that all recordings from the same patient remain in the same split.

3. Groups are partitioned into strata according to the configured `stratify` policy: `af_present` (default), `dataset`, `dataset_af_present`, or `none`.

4. Within each stratum, group identifiers are sorted alphabetically and then shuffled using `random.Random(seed).shuffle()`, where `seed` is the `random_seed` from the configuration.

5. Groups are allocated to train, validation, and test splits using a largest-remainder rule that distributes fractional group counts exactly, ensuring that the configured ratios are honoured as precisely as integer group counts allow.

The same seed applied to the same catalogue snapshot always produces the same split. The split manifest records the `random_seed`, the `config_hash`, the configured ratios, the per-dataset group counts, and a full `assignments` map from recording identifier to split name. It also carries a `split_hash`, computed as the SHA-256 of the canonical split JSON (excluding the hash field itself), which allows the split integrity to be verified independently of the configuration hash.

Per-dataset split caching. Splits are cached per dataset under `s3://ecg-pipeline/benchmarks/cache/splits/{dataset}/{cache_key}/` where the cache key is a hash of the model-independent split inputs: seed, ratios, stratification policy, and the recording IDs, subject IDs, and AF-presence flags of all available recordings. Because model identifiers and metric lists do not enter the cache key, a run that changes only model hyperparameters reuses existing split manifests without recomputation, while a run that adds a new dataset or changes the seed generates a fresh cache entry.

Label policies. The default label policy is `af_vs_nsr`, which retains only windows fully contained inside a single AF or normal sinus rhythm episode and discards boundary-crossing windows and windows with atrial flutter, other, or unknown labels. Three alternative policies are also implemented: `af_vs_non_af` treats all non-AF rhythms as the negative class; `af_or_afl_vs_nsr` groups atrial flutter with AF as the positive class; and `af_or_afl_vs_non_af` combines both expansions. The `af_vs_nsr` default is intentionally the most conservative, producing unambiguous supervised examples at the cost of excluding some rhythm segments from the primary benchmark.

Split artefact persistence. The resulting split manifest is stored as a run artefact under `splits/split_manifest.json` in the run tree rather than written back into the ingestion database, preserving the split-deferral design from Section 3.4.3.

A copy of the assignments is also persisted to the `benchmark_splits` PostgreSQL table for SQL-based analysis.

4.2.4 Feature and Waveform Window Outputs

Spark session and S3 access. Feature generation uses PySpark when distributed processing is available. The `build_features_with_spark()` function creates a SparkSession configured with S3A Hadoop AWS settings pointing at the local MinIO endpoint, using the same environment variables (`AWS_*` or `MINIO_*`) as the rest of the pipeline. Spark reads processed Parquet files directly from `s3a://ecg-pipeline/processed/250hz/{dataset}/` and writes feature outputs to the run artefact tree without intermediate local copies. This allows the same feature-generation code to run on a single-machine Spark local session or a remote cluster by changing only the SparkSession master URL.

Recording manifest. For each dataset, feature generation first writes `features/{dataset}/recording_manifest.parquet`, a recording-level index that joins catalogue metadata (recording ID, subject ID, AF presence, split assignment) with the processed Parquet paths. The evaluator reads this index at evaluation time to join window predictions back to recording identities and split assignments without querying PostgreSQL during the evaluation loop.

Tabular window features. Window rows are enumerated from each recording's annotation sidecar using `_window_rows()`, which steps through the recording at the configured stride and retains windows fully contained in a labelled rhythm episode under the active label policy. For each retained window, RR-interval features are extracted from beat positions within the window boundary and signal statistics are computed by joining the signal Parquet on the sample index range. The resulting tabular feature file written to `features/{dataset}/windows.parquet` contains the following columns per window:

- RR-interval features: `rr_mean`, `rr_std`, `rr_cv` (coefficient of variation), `rr_count`;
- Signal summary features: `signal_mean`, `signal_std`, `signal_min`, `signal_max`, `signal_range`;

- Window metadata: recording identifier, dataset name, split assignment, window start and end sample, selected lead column, binary label, and waveform Parquet path.

Lead selection. The single lead for each window is selected by `select_single_lead()`, which iterates a priority list of preferred lead names (`II`, `MLII`, `I`, `V1` by default) and returns the first matching column found in the recording's channel list, falling back to the first available channel when no preferred lead is present. The selected lead column name is stored in every window row and in the waveform window pointer, so that all downstream consumers use the same channel regardless of which step they originate from.

Waveform window pointers. Feature generation also writes `features/{dataset}/waveform_windows.parquet`, a pointer table containing the waveform Parquet path, selected lead column, start sample, end sample, label, and split assignment for each window. This second artefact allows raw-waveform models to load signal segments lazily without materialising ECG arrays for every run. The separation keeps the tabular and waveform representations consistent in their window boundaries and labels while letting each model family consume only the input form it requires.

4.3 Model Plugins and Evaluation Outputs

The package contains baseline and paper-inspired showcase models. This is consistent with the framework focus: the important implementation contribution is the ability to compare models through a standard API, and establishing a well-documented set of reference detectors is a prerequisite for that comparison.

4.3.1 Model Plugin Interface

Every model plugin follows the `BenchmarkModel` runtime-checkable protocol declared in `models/base.py` and summarised in Table 4.2. Tabular baselines use `scikit-learn`, while waveform models use `torch`. The common `predict_proba` output lets the evaluator compute shared metrics from AF probabilities regardless of how the model was implemented internally.

Table 4.2: Implemented `BenchmarkModel` interface.

Method	Purpose	Parameters	Returns
`fit`	Learn model state from training and validation feature rows. Rule-based models may only apply configuration and persist a loadable artefact.	`train_features`, `val_features`, `config`	Fitted model instance
`predict_proba`	Score each input row with an AF probability; higher values indicate stronger evidence for AF.	`features`	Probability list
`save`	Persist the fitted model artefact to durable storage.	`s3_uri`	None
`load`	Restore a previously saved model artefact.	`s3_uri`	Loaded model instance

Each model also declares a `ModelInfo` object with its `model_id`, `description`, `required_columns`, `input_kind` (`"tabular"` or `"waveform"`), `output_score` (a description of what the probability represents), `reproducibility` (whether results are fully deterministic given the configured seed), `configurable_parameters`, and `requires_training`. Tabular models consume `features/{dataset}/windows.parquet`; waveform models consume `features/{dataset}/waveform_windows.parquet` and load ECG segments lazily from the referenced Parquet files.

The `requires_training` flag distinguishes rule-based detectors from trainable ones. Rule-based models (`requires_training = False`) still implement `fit()`, but the method only applies configuration and persists a reloadable descriptor rather than learning parameters from data. Their training time is therefore zero by construction, while their measured wall time during `fit()` is retained separately in the timing artefacts so that zero training cost is not misread as missing timing data.

PyTorch waveform models are registered in the model registry without importing `torch`. The import occurs only when `fit()` or `predict_proba()` is called. This means the benchmark package installs and validates without a GPU environment, and the optional `[raw]` dependency group in `pyproject.toml` must be installed explicitly by users who intend to exercise waveform models.

4.3.2 Implemented Model Set

Fourteen model plugins are currently registered, spanning three algorithm families and two input kinds, as shown in Table 4.3. The substantive implementations are the nine paper-backed detectors represented by the cited rule-based, handcrafted-feature, boosting, and waveform rows in the table. The other registered plugins—`dummy_prior`, `rr_irregularity_rule`, `logistic_regression_features`, `random_forest_features`, and `raw_cnn_1d`—are intentionally simple smoke-test models. They exercise the registry, configuration, training, prediction, and artefact-writing paths for each broad model group, but they are not presented as literature-backed AF detectors.

Table 4.3: Implemented model plugins. † Rule-based: `requires_training = False`.

Model identifier	Category	Algorithm family	Input
`dummy_prior`	Baseline	Prior-only	tabular
`rr_irregularity_rule`†	DSP	RR coefficient of variation threshold	tabular
`tateno_glass_rr_histogram`†	DSP	RR histogram density [71]	tabular
`lake_moorman_cosen`†	DSP	Sample entropy proxy [72]	tabular
`petrenas_low_complexity`†	DSP	RR + rate fusion [73]	tabular
`logistic_regression_features`	ML (baseline)	Logistic regression (sklearn)	tabular
`random_forest_features`	ML (baseline)	Random forest (sklearn)	tabular
`zabihi_rf_handcrafted`	ML	Random forest on handcrafted features [74]	tabular
`bin_adaboost_tree_ensemble`	ML	AdaBoost tree ensemble	tabular
`goodfellow_xgboost`	ML	Gradient boosting on multiple features [75]	tabular

Continued on next page

Table 4.3: Implemented model plugins (continued).

Model identifier	Category	Algorithm family	Input
`raw_cnn_1d`	Deep Learning (baseline)	1-D convolutional network (PyTorch)	waveform
`xiong_skip_cnn_1d`	Deep Learning	Skip-connection 1-D CNN [76] (PyTorch)	waveform
`zihlmann_crnn_spectrogram`	Deep Learning	CRNN with spectrogram input [77] (PyTorch)	waveform
`cpsc2021_resnet_transformer`	Deep Learning	ResNet + Transformer [78] (PyTorch)	waveform

4.3.3 Evaluation Metrics and Aggregation

Window-level metrics. The evaluator calls `compute_binary_metrics()` for each model × source × target combination. The following metrics are computed from AF probabilities and a configured threshold: area under the receiver operating characteristic curve (`auroc`, computed via the Mann-Whitney U statistic), `average_precision` (area under the precision–recall curve), `accuracy`, `precision`, `recall` (equivalent to `sensitivity`), `specificity`, negative predictive value (`npv`), `f1`, and `balanced_accuracy`. When a metric is undefined for a given evaluation, for example AUROC when a target test set contains only one class, the function returns `None` for that metric rather than a misleading numerical value.

Operating threshold. The first-benchmark operating point is 0.5 unless a source-validation tuning step provides an alternative threshold. The selected threshold is recorded with each metric row so that the operating point used for threshold-dependent metrics is always visible alongside the reported values.

Recording-level aggregation. Recording-level metrics are generated by aggregating window probabilities with one of three configurable policies: `mean` (average of all window probabilities for a recording), `max` (maximum window probability), and `majority_vote` (fraction of windows whose probability meets or exceeds 0.5, treated as 1 if that fraction exceeds 0.5 and 0 otherwise). The recording-level

truth label is derived from window labels with a configurable policy: `any` marks a recording as positive if at least one window is AF-labelled; `majority` requires at least half the windows; and `fraction_threshold` uses a configurable positive fraction.

Timing artefacts. The evaluator records timing artefacts for the benchmarked algorithms. For each trained model it stores the duration of fitting and artefact serialisation; for each evaluation cell it stores inference time and full evaluation time. The inference interval is separated from the broader evaluation envelope so that model latency can be distinguished from feature loading, metric computation, aggregation, and artefact writing. Timing rows include source dataset, target dataset, model identifier, input kind, ISO timestamps, duration, processed-window count, and output artefact URI. This allows downstream analysis to derive per-window latency and to compare models by both accuracy and computational cost.

4.4 Artefact Storage and Reproducibility

All durable benchmark outputs are stored in S3/MinIO under a canonical run tree, while PostgreSQL keeps a queryable catalogue of dataset, recording, run, model, and metric metadata. The hierarchy in Figure 4.1 summarises the implemented layout.

Benchmark PostgreSQL schema. In addition to the data pipeline catalogue tables (`datasets`, `recordings`, `resampled_recordings`), the benchmark package maintains its own schema in PostgreSQL through `catalog.py`. Four tables are created: `benchmark_runs` (run identifier, experiment name, full configuration hash, timestamps, status), `benchmark_splits` (per-recording split assignments linked to a run), `benchmark_models` (model identifier, input kind, source dataset, artefact URI, training timing), and `benchmark_metrics` (per-cell metric rows linking model, source dataset, target dataset, evaluation level, and individual metric values). This relational index allows SQL aggregations across runs without loading any S3 artefact, which is useful for multi-run comparison queries and for the dashboard's Results view.

```
s3://<artifact_bucket>/<artifact_prefix>/<run_id>/
|-- config/
|   |-- resolved_config.json              # resolved configuration and hash
|   `-- benchmark_config_reference.md     # human-readable config snapshot
|-- splits/
|   `-- split_manifest.json               # train/validation/test assignments
|-- features/
|   |-- feature_manifest.json             # index of generated feature datasets
|   `-- <dataset>/
|       |-- recording_manifest.parquet    # recording-level metadata and paths
|       |-- windows.parquet               # tabular features and labels
|       `-- waveform_windows.parquet      # waveform-window pointers
|-- models/
|   `-- <source_dataset>/<model_id>/      # saved model artefacts
|-- predictions/
|   |-- windows/
|   |   `-- <source>/<target>/<model_id>.parquet
|   `-- recordings/
|       `-- <source>/<target>/<model_id>.parquet
|-- metrics/
|   |-- window_metrics.json / .csv        # primary window-level metrics
|   `-- recording_metrics.json / .csv     # secondary recording metrics
|-- matrix/
|   |-- window_benchmark_matrix.csv       # wide window-level matrix
|   `-- recording_benchmark_matrix.csv    # wide recording-level matrix
|-- timings/
|   `-- timings.json / .csv               # training and evaluation timings
`-- RUN_SUMMARY.md                        # run summary and artefact links
```

Figure 4.1: Canonical benchmark run artefact hierarchy in S3/MinIO.

Reusable cache. Split and feature cache entries are stored outside individual run roots under a shared cache prefix. The cache locations are:

`s3://ecg-pipeline/benchmarks/cache/splits/{dataset}/{cache_key}/`
`s3://ecg-pipeline/benchmarks/cache/features/{dataset}/{cache_key}/`

Because the cache key omits model identifiers and metric lists, a run that changes only model configuration reuses the existing partitions and feature files of unmodified datasets, keeping iterative comparison cheap.

Self-documenting run tree. The run tree is self-documenting through generated Markdown files. The configuration reference captures the resolved configuration and the model registry at the time of execution, including available model identifiers, input kinds, descriptions, and configurable parameters. The run summary acts as a front page for the experiment by linking to the resolved configuration, split manifest, feature outputs, model artefacts, predictions, metric files, timing files, and matrices. Because these files are produced by the software workflow,

they reduce the need for manual note-taking and preserve the provenance chain from configuration hash to reported result.

The storage layout is a core part of the reproducibility mechanism. Inspecting a result also reveals the configuration, splits, computational measurements, and artefacts from which it was produced. The implementation therefore avoids treating notebooks or local folders as the authoritative result store. A notebook can demonstrate a run or visualise outputs, but the reproducible state of the experiment lives in the run artefact tree.

4.5 Dashboard and Configuration Builder

The dashboard is implemented as a lightweight local server rather than a large web application. The server uses Python's standard-library `http.server.ThreadingHTTPServer` with a custom `BaseHTTPRequestHandler`, deliberately avoiding web-framework dependencies so that the dashboard runs wherever the Python environment is available without additional installation. Its purpose is to support research workflow tasks: browsing available runs, inspecting metric files and matrices, querying the dataset catalogue, and generating benchmark configurations.

Because the dashboard reads the same S3/MinIO artefact layout and PostgreSQL catalogue as the command-line interface, it is not a separate result system. The dashboard presents five functional areas.

Results view. The Results view combines metric rows with timing rows from the run artefact tree. It can display conventional performance tables and also plot a selected quality metric against prediction time or against combined training-plus-evaluation time. These visualisations expose the practical trade-off between detector quality and computational cost, for example whether a waveform neural model improves AUROC enough to justify its higher training or inference cost compared with an RR-based detector. Timing rows are joined to metric rows using the stored `attach_timing_rows()` service in `dashboard/results.py`.

Explorer view. The Explorer view queries PostgreSQL directly to display the dataset and recording inventory, including recording counts, annotation availabil-

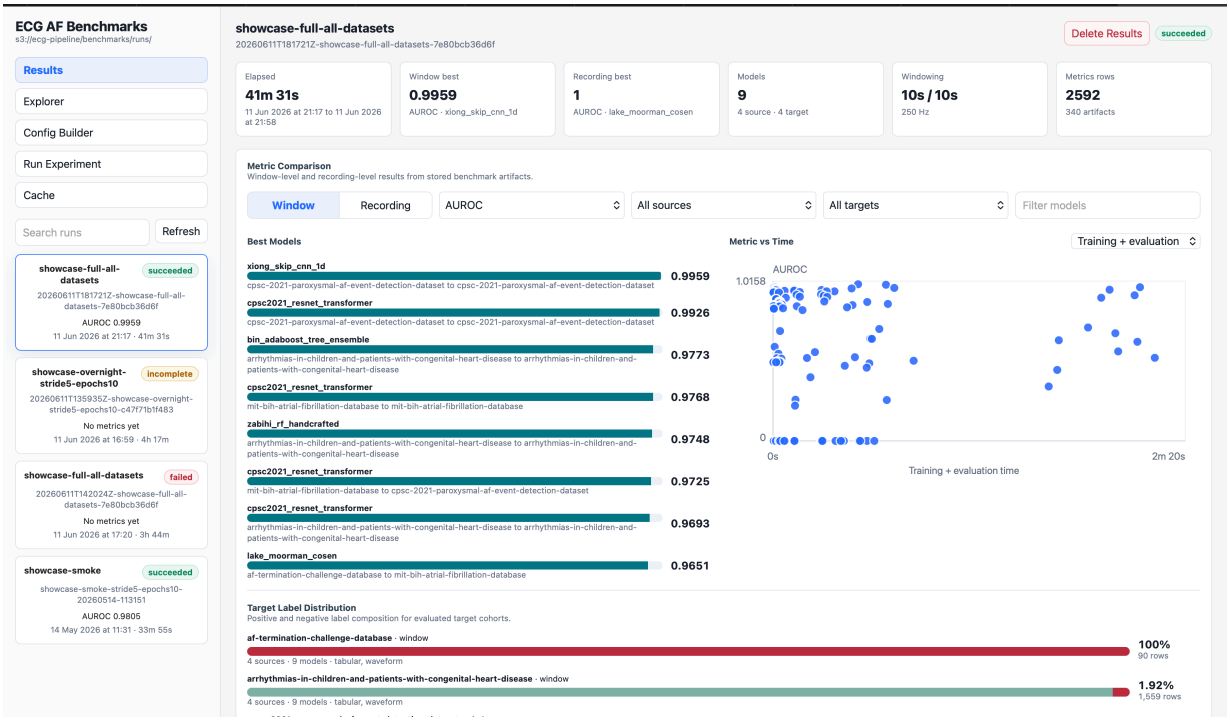


Figure 4.2: Dashboard Results view for a completed benchmark run, combining metric tables with timing-against-quality scatter plots to expose the performance–cost trade-off across model families.

ity, and AF-positive fractions. It also shows the live model registry — all registered model identifiers, input kinds, descriptions, and configurable parameters — so that a researcher can inspect available algorithms before assembling a configuration.

Config Builder. The Config Builder helps assemble YAML benchmark configurations and uses the same validation logic as the command-line workflow. The interface lets the user select datasets, algorithms, training parameters, evaluation behaviour, and run options. Three built-in presets are available: a *Custom* blank form, a *benchmark* preset based on the canonical `benchmark.yaml` example, and a *showcase_smoke* preset for a fast multi-model sanity check. The assembled configuration can be exported as a Markdown launch document, saved to the local filesystem, or uploaded directly to the `benchmarks/configs/` prefix in the S3 bucket so that the configuration artefact lives in the same storage system as the run outputs. This reduces the risk that a dashboard-generated configuration differs semantically from a CLI configuration, while also giving the researcher a downloadable record that can be used to launch or describe the experiment.

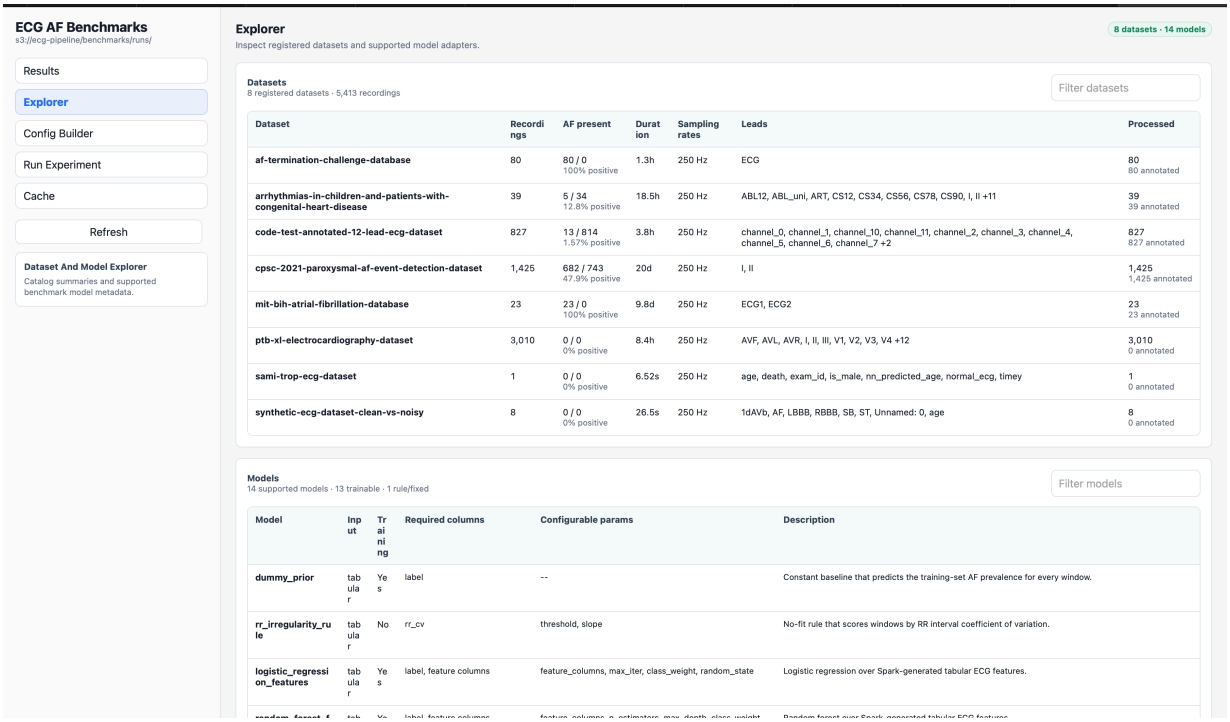


Figure 4.3: Dashboard Explorer view querying the PostgreSQL catalogue to display registered datasets, recording counts, AF-label availability, and the live model registry.

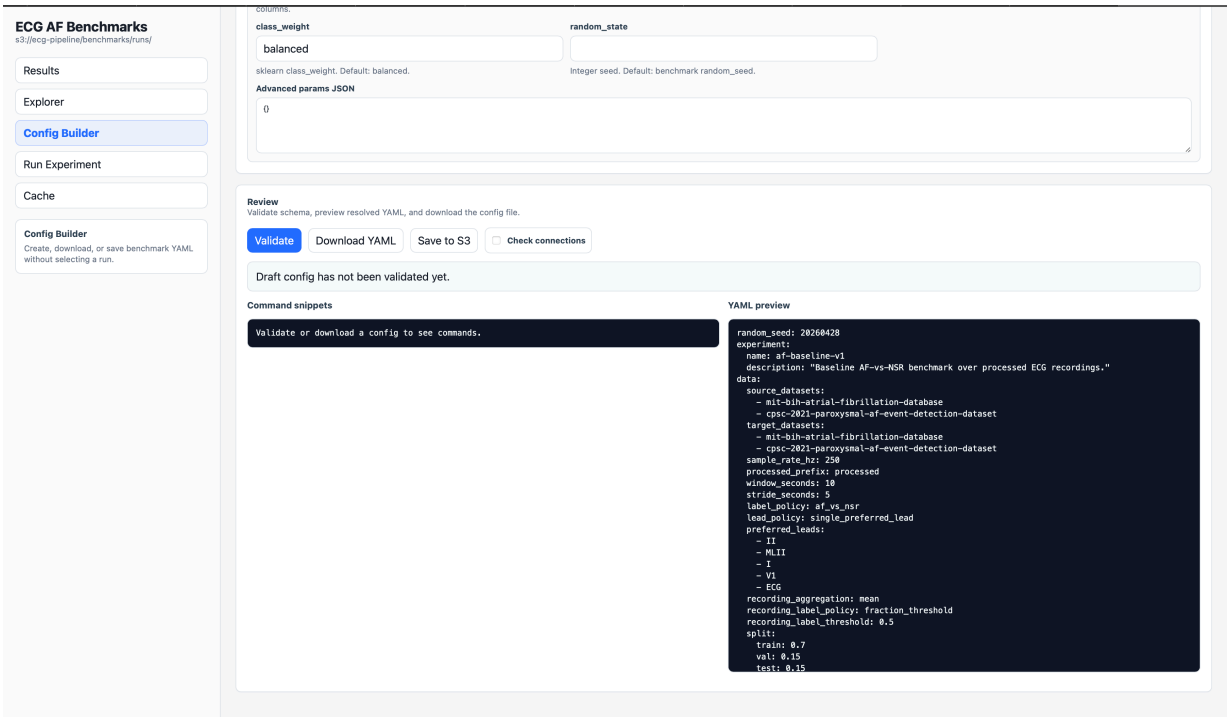


Figure 4.4: Dashboard Config Builder translating dataset, algorithm, and evaluation selections into a validated YAML configuration ready for export or upload to the artefact store.

Run Experiment view. The Run Experiment tab provides a job queue that can launch `ecg-bench run` directly from the browser, either as a local subprocess or via Docker Compose. A researcher can upload or select an existing configuration, submit it as a job, monitor its progress through a JSONL progress event stream, and inspect its stdout and stderr logs — all from the dashboard interface. This view is intended for interactive research sessions where the researcher iterates on configurations without switching between a browser and a terminal.

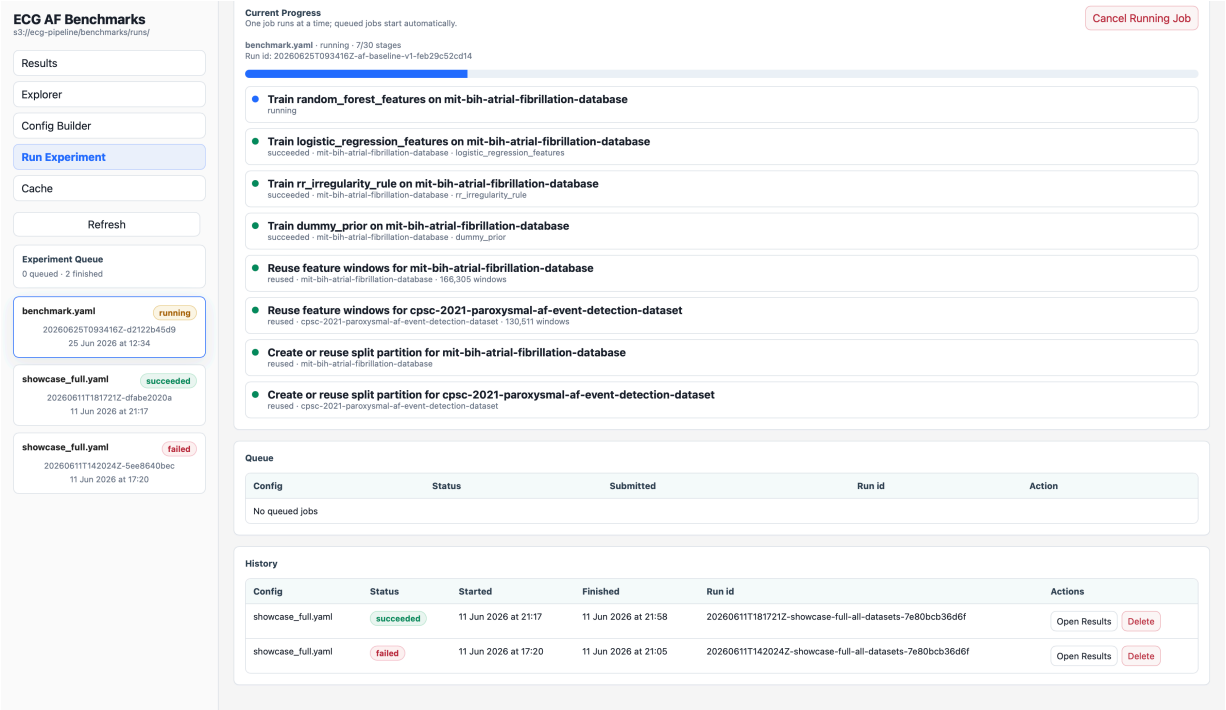


Figure 4.5: Dashboard Run Experiment view for selecting a benchmark configuration, launching an experiment job, and monitoring execution status and logs from the browser.

Cache management. The Cache view lists cached split and feature entries by dataset, shows their cache keys and estimated storage usage, and allows selective deletion of stale cache entries. This is useful when a dataset has been reprocessed and the associated feature cache should be invalidated.

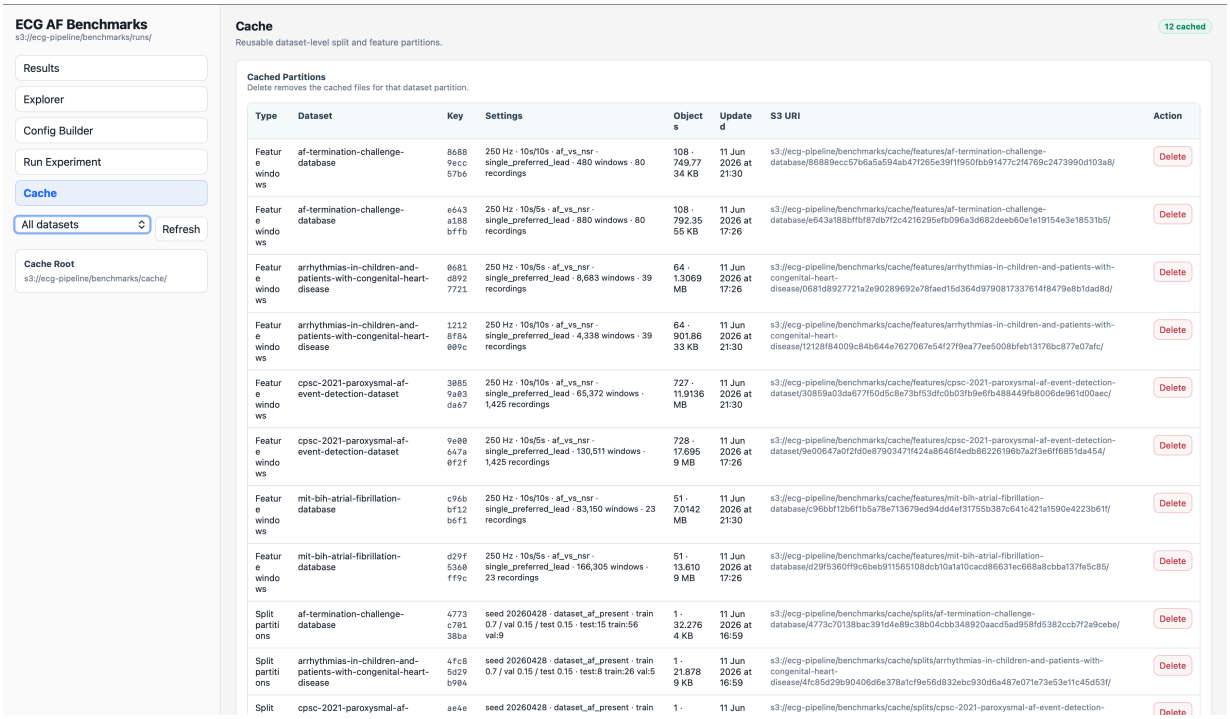


Figure 4.6: Dashboard Cache view listing benchmark split and feature cache entries with their cache keys, estimated storage usage, and controls for selective invalidation.

4.6 Local Infrastructure and Services

The complete framework runtime is defined as a Docker Compose stack, allowing any researcher to reproduce the infrastructure from a single command without installing PostgreSQL, MinIO, or Airflow manually. The primary Compose file (`data-pipeline/docker-compose.yaml`) defines five services.

PostgreSQL 16 provides the relational catalogue for both the data pipeline (`datasets`, `recordings`, `resampled_recordings`) and the benchmark package (`benchmark_runs`, `benchmark_splits`, `benchmark_models`, `benchmark_metrics`). The database is initialised on first start by two SQL scripts that create the Airflow metadata schema and the ECG pipeline schema.

MinIO provides S3-compatible object storage for raw downloads, processed Parquet files, annotation sidecars, and all benchmark run artefacts. A companion **minio-init** service runs once on stack start to create the `ecg-pipeline` bucket and configure the required access policies.

pgAdmin provides a web-based PostgreSQL administration interface for interactive query development and catalogue inspection.

Airflow (webserver, scheduler, and init services) is available as an

optional profile for pipeline orchestration via the three implemented DAGs: `ecg_download_dag`, `ecg_process_register_dag`, and `ecg_pipeline_dag`. The DAGs orchestrate manifest-driven downloads, processing, and registration across datasets with dependency tracking and retry logic. Airflow is enabled by setting the `WITH_AIRFLOW=1` environment variable when starting the stack.

A separate Compose file (`experiments/data_pipeline_test/docker-compose.spark.yaml`) adds a **Jupyter + PySpark** service for interactive EDA and feature development. The Spark master is configured as `local[*]` for single-machine parallelism and is pre-configured with the S3A Hadoop AWS package so that Spark can read from and write to MinIO without additional setup.

The stack is started and stopped with convenience scripts:

```
scripts/start_ecg_af_stack.sh
scripts/stop_ecg_af_stack.sh
```

The default local port assignments are summarised in Table 4.4.

Table 4.4: Default local port assignments for the Docker Compose stack.

Service	Port	Purpose
MinIO API	9000	S3-compatible object storage endpoint
MinIO Console	9001	MinIO web administration UI
PostgreSQL	5432	Relational catalogue
pgAdmin	5050	Database administration UI
Jupyter (Spark)	8889	Interactive notebooks and PySpark EDA
Dashboard	8765	Benchmark dashboard and Config Builder

Because the entire infrastructure runs locally via Docker Compose, the framework can operate in a fully offline environment after the initial Docker image pull. This is relevant for research settings where external network access to cloud object stores is restricted or where all data must remain on-premises.

4.7 Automated Test Coverage

The benchmark package ships with a `pytest` suite under `py_ecg_benchmark_pkg/tests/` that encodes the core invariants required for reproducible

experimentation. Table 4.5 lists the test modules and the properties they verify.

Table 4.5: Automated test modules in `py_ecg_benchmark_pkg/tests/`.

Test module	Properties verified
`test_config.py`	Default resolution, hash stability across equivalent configs, `random_seed` required, model ID registry membership, environment variable names
`test_splits.py`	Reproducibility across calls with the same seed, subject-safe grouping (no subject appears in two splits), stratification balance, recording-level fallback when subject IDs are absent
`test_features.py`	Lead selection priority order, RR coefficient of variation computation in window features
`test_labels.py`	Window retention under all four label policies, episode boundary exclusion, rhythm canonicalisation from pipeline JSON
`test_metrics.py`	AUROC and F1 numerical correctness, recording aggregation under all three policies, matrix pivot shape and column names
`test_models.py`	All model families instantiate and conform to the `BenchmarkModel` protocol, sklearn column contracts, torch models register without importing `torch`
`test_artifacts.py`	`ArtifactLayout` URI construction, run-ID format, config hash embedding
`test_cache.py`	Cache keys are independent of model identifiers and metric lists; sensitive to seed, window length, stride, and label policy
`test_workflow.py`	Feature build summary structure, evaluation row limits and schema, storage environment variable handling
`test_dashboard.py`	Dashboard service layer, timing-row attachment to metric rows, config API validation, runner job lifecycle
`test_runner.py`	Experiment runner job queue, Docker Compose command factory, Java environment detection for Spark

The suite is executed with:

```
uv run --project py_ecg_benchmark_pkg pytest
```

The tests for `test_splits` and `test_config` are particularly critical for the reproducibility claims: they assert that the same seed and catalogue snapshot always produce the same split assignments, and that the configuration hash changes if and only if a methodologically significant parameter changes. These properties are preconditions for the cross-dataset comparison results reported in Section 5.1, especially the transfer results summarised in Table 5.2.

4.8 Chapter Summary

This chapter described the implemented framework as a reproducible ECG-AF benchmarking artefact. The data pipeline prepares heterogeneous ECG datasets through manifest-driven download (supporting four provider strategies: PhysioNet S3, Zenodo, Figshare, and direct URL), a `FormatDispatcher` that routes WFDB, HDF5, CSV, and MAT files to format-specific readers, polyphase resampling to 250 Hz, self-describing Parquet outputs with Snappy compression, structured JSON annotation sidecars, and idempotent PostgreSQL registration across three tables with UPSERT semantics.

The benchmark package creates hash-traceable runs with a SHA-256 configuration hash embedded in the run identifier and in three storage locations, a five-step deterministic split algorithm with subject- safe grouping and largest-remainder allocation, four label policy variants with conservative `af_vs_nsr` as the default, Spark-based feature generation producing nine named feature columns per window, per-dataset split and feature caching keyed independently of model configuration, and a standard model API backed by 14 registered plugins across tabular-rule, tabular-ML, and PyTorch-waveform families. Evaluation computes nine metrics at window and recording level for every model $\times$ source $\times$ target combination and records separate inference and training timing rows. All outputs are stored in a self-documenting S3/MinIO run tree and indexed in a four-table PostgreSQL benchmark schema.

The dashboard, implemented on Python's standard-library HTTP server, provides five views: Results with timing-integrated scatter plots, Explorer for live dataset and model-registry inspection, Config Builder with validation and preset-based authoring, Run Experiment for browser-launched job execution, and Cache management. The entire infrastructure runs as a Docker Compose stack that a researcher can start with a single script, covering PostgreSQL 16, MinIO, pgAdmin, optional Airflow orchestration, and a Jupyter + Spark service for interactive development. A `pytest` suite of eleven test modules encodes the split-determinism, hash-stability, metric-correctness, and model-protocol invariants on which the reproducibility claims depend.

Chapter 5

Results, Discussion, and Practical Implications

This chapter reports the outcome of running the framework described in Chapter 4, interprets what that outcome shows, and explains the practical value of the benchmark. As established in Chapters 3 and 4, the thesis contribution is the experimental environment in which detectors are compared; the results below are evidence that this environment produces fair, traceable comparisons, and should be read as such—not as a definitive clinical ranking of AF algorithms.

5.1 Demonstration Run and Reported Metrics

To demonstrate the framework end-to-end, a single benchmark run was executed over the four implemented datasets (Table 4.1) using the default configuration: 10 s windows with a 5 s stride, 250 Hz single-lead input, conservative `af_vs_nsr` labelling, 70/15/15 splits stratified by AF presence, a 0.5 decision threshold, and the 9 main registered model plugins. Each model was trained once per source dataset and evaluated on every target dataset, producing the model $\times$ source $\times$ target structure defined in Section 3.4.2. All results reported in this chapter can be reproduced by running the `showcase-overnight-stride5-epochs10` benchmark configuration included in the project repository.

Window-level performance. Table 5.1 reports window-level metrics for the within-dataset case (source equals target), averaged across datasets. These are the primary benchmark outputs because they are closest to the supervised labels. The three PyTorch models were each trained for 10 epochs with a maximum of 4 096 training windows on CPU, constituting a proof-of-concept run rather than a fully converged model; their results should therefore be treated as indicative lower bounds on achievable performance.

Table 5.1: Within-dataset window-level results (mean across implemented datasets) at threshold 0.5. † Trained for 10 epochs, $\leq$4 096 windows, CPU only.

Model	AUROC	F1	Sens.	Spec.	AP
`tateno_glass_rr_histogram`	0.8656	0.6619	0.7620	0.7868	0.6835
`lake_moorman_cosen`	0.9308	0.7969	0.8710	0.8422	0.8030
`petrenas_low_complexity`	0.8987	0.7326	0.8900	0.8006	0.7276
`zabihi_rf_handcrafted`	0.9231	0.6766	0.6945	0.8451	0.8270
`bin_adaboost_tree_ensemble`	0.9319	0.7920	0.7817	0.8355	0.8547
`goodfellow_xgboost`	0.9382	0.8263	0.8234	0.8472	0.8633
`xiong_skip_cnn_1d`†	0.9696	0.8315	0.9659	0.9095	0.8715
`zihlmann_crnn_spectrogram`†	0.9371	0.8055	0.8644	0.9275	0.7908
`cpsc2021_resnet_transformer`†	0.9905	0.7548	0.9725	0.8395	0.9124

Cross-dataset transfer. The off-diagonal cells of the evaluation matrix measure transfer to an unseen dataset and are the most informative output of the framework, because they expose whether a detector's performance depends on its training source. Table 5.2 summarises the change from within-dataset to cross-dataset evaluation for each model family; a large drop indicates source-specific tuning rather than generalisable detection.

Table 5.2: Within- vs. cross-dataset AUROC by model family (mean across all source/target pairs).

Model family	Within-dataset	Cross-dataset	ΔAUROC
Rule-based (RR)	0.8984	0.8975	-0.0009
Tabular ML	0.9310	0.7043	-0.2267
Raw-waveform (PyTorch)	0.9657	0.6412	-0.3245

Table 5.3: Within- vs. cross-dataset AUROC by model (mean across all source/target pairs).

Model	Within-dataset	Cross-dataset	ΔAUROC
Rule-based (RR)			
`tateno_glass_rr_histogram`	0.8656	0.8656	0.0000
`lake_moorman_cosen`	0.9308	0.9308	0.0000
`petrenas_low_complexity`	0.8987	0.8961	−0.0026
Tabular ML			
`zabihi_rf_handcrafted`	0.9231	0.6805	−0.2425
`bin_adaboost_tree_ensemble`	0.9319	0.7269	−0.2050
`goodfellow_xgboost`	0.9382	0.7055	−0.2327
Raw-waveform (PyTorch)			
`xiong_skip_cnn_1d`	0.9696	0.6758	−0.2938
`zihlmann_crnn_spectrogram`	0.9371	0.6175	−0.3196
`cpsc2021_resnet_transformer`	0.9905	0.6305	−0.3600

Computational cost. Table 5.4 reports training and per-window inference time, which the framework treats as a first-class outcome. In long-term monitoring, a method that marginally improves a quality metric may still be impractical if it is too slow to score multi-day recordings.

Table 5.4: Training time and per-window inference latency by model.

Model	Algorithm family	Train time (s)	Inference (μs/window)
`tateno_glass_rr_histogram`	RR histogram	0 (by construction)	0.35
`lake_moorman_cosen`	Entropy proxy	0 (by construction)	0.76
`petrenas_low_complexity`	RR + rate fusion	0 (by construction)	0.59
`zabihi_rf_handcrafted`	Handcrafted-feature ML	2.4	34.13
`bin_adaboost_tree_ensemble`	Boosted trees	7.3	6.03
`goodfellow_xgboost`	Gradient boosting	16.8	2.05
`xiong_skip_cnn_1d`	Skip-connection CNN	68.2	11970.51
`zihlmann_crnn_spectrogram`	Spectrogram CRNN	148.8	5888.72
`cpsc2021_resnet_transformer`	ResNet + Transformer	2916.5	11582.19

These tables are produced directly from the stored metric and timing artefacts described in Section 4.4, so every reported value can be traced back to the configuration hash, split manifest, and predictions that generated it.

5.2 Interpretation

The run confirms operational feasibility: the design from Chapter 3 executes as a complete workflow, from source recordings through standardisation, deterministic splits, feature generation, and model evaluation, to inspectable metrics and artefacts. Beyond feasibility, the demonstration supports three observations.

First, dataset heterogeneity is reduced without hiding provenance. The pipeline does not pretend that differences in sampling rate, file layout, and annotation style disappear; it places each dataset behind a manifest and adapter, writes a standardised representation, and keeps the source decisions visible. When results later disagree, the experimenter can inspect exactly how each recording entered the benchmark. Figure 4.3, introduced in Section 4.5, shows this audit layer in the dashboard.

Second, comparison becomes meaningful because the benchmark controls the experimental contract. Each detector fits the same probability-output interface and is scored by common logic over identical windows and splits, so differences are easier to attribute to the algorithm than to inconsistent evaluation code. The cross-dataset results in Table 5.2 are only interpretable because the within- and cross-dataset cases are produced by the same workflow.

Third, the timing outputs in Table 5.4 change how the quality metrics should be read. A simple RR-based detector that is fast and reproducible may be preferable for screening research even at lower AUROC, while a waveform model is justified only if its accuracy gain offsets its cost. Reporting cost alongside quality connects the benchmark to real deployment constraints. Figure 4.2, introduced in Section 4.5, shows this combined, cost-aware view for a completed run. Figure 4.4, also introduced in Section 4.5, shows that experiment setup is captured as explicit YAML rather than hidden in the interface, so any run can be stored, reviewed, and re-executed.

5.3 Practical Value and Intended Use

The benchmark bridges algorithm development and real-world evaluation. Many AF detectors are published as isolated methods, but researchers and clinical-technology teams usually need to answer a different question: which method stays

reliable when the data source, label policy, recording duration, or computational budget changes? Because the framework makes each of these an explicit, configurable axis, a user can change one factor at a time and observe whether a result moves because of the model, the dataset, the label policy, or the split. This keeps negative or inconclusive results informative: the failing configuration remains documented and reusable.

Table 5.5 summarises the intended user groups. The same view also marks the boundary of the contribution introduced in Chapter 3: the framework supports scientific comparison and pre-clinical evidence generation, and is not a diagnostic system. If later connected to a clinical workflow, it should be treated as a research and evaluation layer that informs which detectors merit more formal validation.

Table 5.5: Intended users of the framework and expected benefits.

User group	**Reason to use the framework**	**Expected benefit**
Scientists and researchers	Compare AF methods under shared dataset, split, label, and metric policies.	Reproducible evidence and clearer explanations for performance differences.
Algorithm developers	Add new model plugins or feature builders and evaluate against existing baselines.	Faster iteration without rewriting the data-preparation and evaluation stack.
Clinical research teams	Test candidate detectors on retrospective long-term ECG collections before prospective study.	Understanding of which tools may support triage or workload reduction.
Dataset curators	Check whether recordings and annotations convert into benchmark-ready form.	Clearer documentation of dataset coverage and processing decisions.

Continued on next page

Table 5.5: Intended users of the framework and expected benefits (continued).

User group	Reason to use the framework	Expected benefit
Students and educators	Study the full lifecycle from raw ECG to metrics and artefacts.	A concrete teaching example for reproducible biomedical ML.

The connection to real-life monitoring follows from the growing volume of long-term ECG data: manual review of every window is slow and costly, while fully automatic interpretation is risky without careful validation. A reproducible benchmark sits between these extremes by deciding which detectors are promising enough to study further and under which constraints. Concretely, it supports retrospective algorithm selection on data resembling a target population, workload-oriented triage research (comparing detectors by scoring speed, false-positive load, and sensitivity), and dataset-generalisation analysis via the cross-dataset matrix.

5.4 Limitations on Interpreting the Current Results

The limitations below concern how far the present results can be read; the corresponding development responses are deferred to the future-work roadmap in Section 6.3 to avoid duplication.

The reported metrics come from a single demonstration run on a deliberately small corpus. They are sufficient to show that the workflow produces traceable, comparable outputs, but they do not constitute a clinical ranking: the run lacks repeated trials, confidence intervals, and formal statistical comparison between detectors, so observed differences should be treated as illustrative rather than conclusive.

The conservative `af_vs_nsr` labelling that makes the comparison clean also narrows what the numbers describe. Boundary-crossing windows, atrial flutter, ectopy, noisy segments, and clinically meaningful transitions are excluded from the primary task, so the reported performance reflects the cleanest sub-problem rather than the full difficulty of real recordings. Results under the broader label policies are therefore not interchangeable with the default ones.

Finally, the dashboard supports interpretation but has not been evaluated through usability testing with cardiologists or external teams, so claims about reviewer workload remain prospective. These constraints bound the current evidence without undermining the framework's purpose, which is to make the next, larger round of comparison easier to run and audit.

5.5 Chapter Summary

This chapter reported a demonstration run and used it to show that the framework yields fair, cost-aware, and traceable AF comparisons. The within- and cross-dataset metrics, produced by identical benchmark logic, illustrate how the environment connects data standardisation, reproducible configuration, shared evaluation, and durable artefacts. The current evidence is bounded by a single small run and conservative labelling, which sets up the future-work roadmap of Chapter 6.

Chapter 6

Conclusion and Future Work

This thesis approached reproducible AF benchmarking as an infrastructure problem—the design of the environment in which detectors are compared. This chapter consolidates the contribution, answers the research question directly, and sets out the extensions that would turn the demonstrated framework into stronger scientific evidence.

6.1 Summary of Contributions

The thesis delivered, as a single coherent system, four connected outputs: a manifest-driven data pipeline that standardises heterogeneous ECG recordings while preserving provenance; a benchmark methodology defining deterministic splits, conservative label policies, a shared model interface, window- and recording-level metrics, and computational-cost reporting; an implemented software artefact (command-line workflows, 14 model plugins, structured run outputs, artefact storage, and a dashboard) that realises that methodology; and a demonstration run, reported in Section 5.1, showing the system executes end-to-end and produces traceable cross-dataset comparisons. The distinguishing feature across all four is that experimental assumptions are expressed as explicit, hashed configuration rather than hidden in separate scripts.

6.2 Answer to the Research Question

The research question asked how a reproducible data pipeline and benchmarking framework can be designed to support fair, scalable, and transparent comparison of AF detection methods in long-term ECG monitoring. The work answers it through four mechanisms, each demonstrated rather than only proposed.

Fairness is achieved by a shared experimental contract: every detector emits probabilities in a common format and is scored by identical benchmark logic, so the within- and cross-dataset results in Chapter 5 are directly comparable. Scalability is achieved by separating dataset preparation and feature generation from individual model code, so new datasets enter through manifests and adapters and cached preparation can be reused. Transparency is achieved by storing configurations, splits, label decisions, metrics, timings, and artefacts in a durable, hash-linked form. Usefulness is achieved by making those outputs human-readable through the dashboard and configuration builder.

Taken together, these show that reproducibility is not a single feature but the combined effect of traceable preparation, explicit configuration, deterministic execution, shared evaluation, and preserved outputs — and that this combination can be implemented as one working framework.

6.3 Future Improvements

The future work below responds directly to the interpretation limits identified in Section 5.4, turning each constraint into a concrete development direction.

The highest priority is to move from a single demonstration run to defensible evidence: repeated trials with multiple seeds, confidence intervals, and statistical comparison between detectors, together with a fuller cross-dataset matrix over more source/target pairs. This is what would convert the current feasibility demonstration into claims about model behaviour.

The second priority is broader dataset coverage. The appendix survey identifies many candidate resources beyond the four implemented datasets; adding manifests, readers, annotation adapters, and validation checks would extend the benchmark across more devices, durations, populations, and rhythm classes, which is the precondition for any general performance claim.

The third priority is richer signal-quality handling. Future versions should compute signal-quality indicators, expose quality distributions in the dashboard, and let experiments compare inclusion, exclusion, and down-weighting policies, so that algorithm failure can be distinguished from poor input quality — a distinction the current conservative labelling cannot make.

Finally, the model library should grow to include more classical RR detectors, tabular models, raw-waveform networks, and hybrid DSP–ML approaches, each documented with input requirements and timing outputs so that accuracy and cost continue to be reported together.

6.4 Conceptual Mock-ups for Future Updates

To communicate the longer-term vision, Table 6.1 describes conceptual mock-ups of future interface and workflow updates. These are proposed directions, not completed features; each extends the guiding idea of the thesis — making hidden decisions visible.

Table 6.1: Conceptual mock-ups for future benchmark updates.

Future update	**Mock-up behaviour**	**Purpose**
Dataset coverage panel	Lists each dataset, processing status, annotation support, licensing notes, and failed records.	Shows what data are actually benchmark-ready and what remains incomplete.
Run comparison workspace	Aligns metrics, timing, confusion matrices, and configuration diffs across selected runs.	Clarifies why one detector behaves differently from another.
Signal-quality explorer	Plots noise indicators, excluded windows, and quality distributions by dataset or recording.	Connects model errors to ECG quality rather than treating all windows as equal.
Model card export	Generates a short report with data sources, label policy, settings, metrics, timing, and limitations.	Supports publication, peer review, and internal audit.

Continued on next page

Table 6.1: Conceptual mock-ups for future benchmark updates (continued).

Future update	Mock-up behaviour	Purpose
Clinical-review simulation	Shows high-probability AF windows beside the ECG segment and reviewer notes.	Demonstrates how outputs might support future triage research without claiming diagnosis.
Federated evaluation mode	Sites run the same configuration locally and share only aggregate metrics.	Lets private ECG collections contribute evidence without centralising sensitive data.

6.5 Final Remarks

AF detection in long-term ECG monitoring is both a clinical and a reproducibility challenge: reliable evidence depends on how data are prepared, how labels are defined, how splits are generated, how metrics are computed, and how results are preserved—as much as on the algorithms themselves. This thesis contributes a framework that brings those decisions into one inspectable workflow, turning algorithm comparison into a documented experiment rather than a collection of isolated scripts.

Its value is realistic and staged. The framework does not diagnose patients or replace expert interpretation; it provides the reproducible comparison layer that should exist before such claims are made. By making data preparation, evaluation, timing, and artefact storage explicit, it gives future teams a principled basis for deciding which detectors deserve the more expensive, clinically demanding validation that real deployment requires.

Appendix A

Candidate Dataset Survey

This appendix records the ECG datasets considered during the curation stage described in Chapter 3. It documents dataset discovery and selection decisions, including datasets that were not used in the current thesis implementation. Exclusion decisions were based on annotation availability, task alignment, access practicality, storage requirements, population fit, and the limited scope of the first AF-vs-NSR benchmarking implementation.

A.1 Datasets Not Selected for the Current Implementation

- **ECG in High-Intensity Exercise Dataset**. Source: `https://zenodo.org/records/5727800`. Not selected because no AF annotations were identified, so the dataset does not support AF-vs-NSR supervision in the current benchmark.
- **Sami-Trop ECG Dataset**. Source: `https://zenodo.org/records/4905618`. Not selected because no AF annotations were identified; the dataset is oriented toward Chagas-disease ECG analysis rather than AF-labelled benchmarking.
- **ISP ECG Delineation Dataset**. Source: `https://zenodo.org/records/14679837`. Not selected because the available labels support ECG waveform delineation rather than AF episode or rhythm classification.
- **CODE-test: An annotated 12-lead ECG dataset**. Source: `https://ze`

nodo.org/records/3765780. Not selected for the first implementation because it has AF-related diagnostic labels but no beat annotations were identified, and it may overlap with the CODE-15% ECG Dataset.

- **Arrhythmias in Children and Patients with Congenital Heart Disease**. Source: https://physionet.org/content/leipzig-heart-center-ecg/1.0.0/. Not selected because its paediatric and congenital-heart-disease population differs from the adult long-term AF monitoring focus of the first benchmark.

- **CPSC 2021 Varying Dimensions in Electrocardiography**. Source: https://physionet.org/content/challenge-2021/1.0.3/. Not selected for automated ingestion in the current implementation because access requires AWS credentials or manual download handling.

- **Icentia11k Continuous ECG Database**. Source: https://physionet.org/content/icentia11k-continuous-ecg/1.0/. Not selected because the full resource is approximately 1.1 TB, exceeding the practical storage and processing scope of the thesis implementation.

A.2 Datasets Retained as Candidates or References

- **CODE-15% ECG Dataset (Brazil)**. Source: https://zenodo.org/records/4916206. Retained as a candidate for future multi-lead diagnostic-label experiments because it contains 12-lead ECGs with diagnostic statements that include AF-related labels.

- **IRIDIA-AF Database**. Source: https://zenodo.org/records/8405941. Retained as an AF-specific wearable ECG candidate and as a useful Zenodo-hosted dataset for provider-ingestion testing.

- **MIT-BIH Atrial Fibrillation Database**. Source: https://physionet.org/content/afdb/1.0.0/. Retained as a primary AF reference dataset because it provides rhythm annotations and is widely used in AF detection research.

- **Long-Term Atrial Fibrillation Database (LTAFDB)**. Source: `https://physionet.org/content/ltafdb/1.0.0/`. Retained as a long-duration AF monitoring candidate because it aligns with the thesis emphasis on extended ECG recordings.

- **SHDB-AF Database (Japan)**. Source: `https://physionet.org/content/shdb-af/1.0.1/`. Retained as a cross-population AF Holter candidate for future framework extension.

- **PhysioNet / CinC Challenge 2017 (AF Classification)**. Source: `https://physionet.org/content/challenge-2017/1.0.0/`. Retained as a candidate benchmark dataset with short single-lead AF classification labels, although its task differs from the main long-term workflow.

- **CPSC 2021 Paroxysmal AF Event Detection Dataset**. Source: `https://physionet.org/content/cpsc2021/1.0.0/`. Retained as a candidate for event-level AF experiments and future label-adapter development.

- **MIT-BIH Arrhythmia Database**. Source: `https://physionet.org/content/mitdb/1.0.0/`. Retained as a supporting arrhythmia reference, but not prioritised as a primary AF dataset because it is not AF-specific.

- **Chapman–Shaoxing 12-Lead ECG Database**. Source: `https://figshare.com/collections/ChapmanECG/4560497`. Retained as a Figshare-hosted candidate because it provides multi-label rhythm and condition annotations for future multi-label rhythm-classification work.

- **SPH ECG Dataset (Shandong Provincial Hospital)**. Source: `https://figshare.com/articles/dataset/SPH_dataset/21553885`. Retained as a Figshare-hosted candidate because it provides AHA-standardised diagnostic statements for future diagnostic-label experiments.

Overall, the curation log shows that dataset selection was constrained by both whether a dataset was publicly visible and whether its annotations, access method, scale, and target population matched the first reproducible AF benchmarking workflow. The first framework release therefore prioritised a small representative corpus while preserving these candidates for future extension.

Appendix B

Artefact Compatibility Notes

This appendix records implementation details that are preserved for backward compatibility or for consumers of the artefact layout that pre-date the current naming conventions. These details are correct descriptions of the deployed implementation but are too granular for the main Chapter 4 discussion.

B.1 Canonical Baseline Configuration

The baseline demonstration run used the YAML configuration below. It is included here to preserve the full experiment contract without interrupting the implementation narrative in Chapter 4.

```
random_seed: 20260428

experiment:
  name: af-baseline-v1
  description: Baseline AF-vs-NSR benchmark over processed ECG recordings.

data:
  # Source datasets train independent model instances.
  source_datasets:
    - mit-bih-atrial-fibrillation-database
    - cpsc-2021-paroxysmal-af-event-detection-dataset

  # Target datasets are evaluated with each trained source model.
  target_datasets:
    - mit-bih-atrial-fibrillation-database
    - cpsc-2021-paroxysmal-af-event-detection-dataset

  sample_rate_hz: 250
  processed_prefix: processed
  window_seconds: 10
  stride_seconds: 5
  # Supported: af_vs_nsr, af_vs_non_af, af_or_afl_vs_nsr, af_or_afl_vs_non_af.
  label_policy: af_vs_nsr
```

```
  lead_policy: single_preferred_lead
  preferred_leads: [II, MLII, I, V1, ECG]

  # Recording-level outputs are secondary; window-level metrics are primary.
  recording_aggregation: mean
  recording_label_policy: fraction_threshold
  recording_label_threshold: 0.5

  split:
    train: 0.70
    val: 0.15
    test: 0.15
    stratify_by: dataset_af_present

storage:
  artifact_bucket: ecg-pipeline
  artifact_prefix: benchmarks/runs
  cache_prefix: benchmarks/cache
  source_bucket: ecg-pipeline

reuse:
  enabled: true
  splits: true
  features: true
  mode: auto

database:
  host_env: POSTGRES_HOST
  port_env: POSTGRES_PORT
  db_env: POSTGRES_DB
  user_env: POSTGRES_USER
  password_env: POSTGRES_PASSWORD

spark:
  master_env: SPARK_MASTER
  app_name: ecg-af-benchmark

models:
  - id: dummy_prior
  - id: rr_irregularity_rule
    params:
      threshold: 0.12
      slope: 12.0
  - id: logistic_regression_features
    params:
      feature_columns: &tabular_features
        - rr_count
        - rr_mean
        - rr_std
        - rr_cv
        - signal_mean
        - signal_std
        - signal_min
        - signal_max
        - signal_range
      max_iter: 1000
      class_weight: balanced
  - id: random_forest_features
    params:
      feature_columns: *tabular_features
```

```
      n_estimators: 200
      class_weight: balanced

metrics:
  - n
  - positive
  - negative
  - auroc
  - average_precision
  - accuracy
  - f1
  - sensitivity
  - specificity
```

B.2 Metric and Matrix Compatibility Aliases

The canonical metric artefacts written by the benchmark evaluator use the naming scheme introduced in Section 4.4:

- `metrics/window_metrics.json` and `metrics/window_metrics.csv` for window-level metrics;
- `metrics/recording_metrics.json` and `metrics/recording_metrics.csv` for recording-level metrics;
- `matrix/window_benchmark_matrix.csv` and `matrix/recording_benchmark_matrix.csv` for the wide benchmark matrices.

In addition, the evaluator writes compatibility aliases for consumers that expect the earlier flat naming convention:

- `metrics/metrics.json` and `metrics/metrics.csv` mirror the window-level metric files (`metrics/window_metrics.*`);
- `matrix/benchmark_matrix.csv` mirrors the window-level benchmark matrix (`matrix/window_benchmark_matrix.csv`).

These aliases are written as separate objects with identical content. They will be removed in a future release once all consumers have been updated to the canonical names. New tooling and analysis scripts should reference the canonical paths rather than the aliases.

B.3 S3 Object Metadata Fields

The `config/resolved_config.json` object carries the key–value pair `config-hash: {full_sha256_hex}` as S3 object metadata (not as a field inside the JSON body). This allows the configuration hash to be retrieved via a `HeadObject` call without downloading and parsing the full configuration file, which is useful in tooling that validates run identity without reading the complete configuration.

B.4 Legacy Run Identifier Format

The current run identifier format is:

```
{YYYYMMDDTHHMMSSZ}-{sanitized_experiment_name}-{config_hash[:12]}
```

Older runs produced before the configuration hash was introduced used a shorter format without the hash suffix:

```
{YYYYMMDDTHHMMSSZ}-{sanitized_experiment_name}
```

The dashboard and matrix commands detect this shorter format by checking whether the run directory name contains a third hyphen-delimited token of exactly 12 hexadecimal characters, and fall back to treating the run as a legacy run when the check fails. Legacy runs can still be browsed in the dashboard Results and Explorer views, but configuration-hash verification is unavailable for them.

Bibliography

[1] Jinho Park, Sangwook Lee, and Moongu Jeon. Atrial fibrillation detection by heart rate variability in Poincaré plot. *BioMedical Engineering OnLine*, 8: 38, 2009. doi: 10.1186/1475-925X-8-38.

[2] S. Dash, K. H. Chon, S. Lu, and E. A. Raeder. Automatic real time detection of atrial fibrillation. *Annals of Biomedical Engineering*, 37(9):1701–1709, 2009. doi: 10.1007/s10439-009-9740-z.

[3] Gari D. Clifford, Chengyu Liu, Benjamin Moody, H. Lehman Li-wei, Ikaro Silva, Qiao Li, Alistair E. W. Johnson, and Roger G. Mark. AF classification from a short single lead ECG recording: The PhysioNet/Computing in Cardiology Challenge 2017. In *Computing in Cardiology*, volume 44, pages 1–4, 2017. doi: 10.22489/CinC.2017.065-469.

[4] Masud Shah Jahan, Marjan Mansourvar, Sadasivan Puthusserypady, Uffe Kock Wiil, and Abdolrahman Peimankar. Short-term atrial fibrillation detection using electrocardiograms: A comparison of machine learning approaches. *International Journal of Medical Informatics*, 163:104790, 2022. doi: 10.1016/j.ijmedinf.2022.104790.

[5] Chao Xie, Zirui Wang, Chenxi Yang, Jianhe Liu, and Hao Liang. Machine learning for detecting atrial fibrillation from ECGs: Systematic review and meta-analysis. *Reviews in Cardiovascular Medicine*, 25(1):8, 2024. doi: 10.31083/j.rcm2501008.

[6] Awni Y. Hannun, Pranav Rajpurkar, Masoumeh Haghpanahi, Geoffrey H. Tison, Codie Bourn, Mintu P. Turakhia, and Andrew Y. Ng. Cardiologist-level arrhythmia detection and classification in ambulatory electrocardio-

grams using a deep neural network. *Nature Medicine*, 25(1):65–69, 2019. doi: 10.1038/s41591-018-0268-3.

[7] Chao Xie, Zirui Wang, Chenxi Yang, Jianhe Liu, and Hao Liang. Machine learning for detecting atrial fibrillation from ECGs: Systematic review and meta-analysis. *Reviews in Cardiovascular Medicine*, 25(1):8, 2024. doi: 10.31083/j.rcm2501008.

[8] Jiahao Zou, Can Liu, Yatao Zhang, Yue Zhao, Wentao Wang, Xiaoyan Luo, Yansheng Xu, Wenxin Wang, and Mingfeng Jiang. A generalizable and robust deep learning method for atrial fibrillation detection from long-term electrocardiogram. *Biomedical Signal Processing and Control*, 89:105720, 2024. doi: 10.1016/j.bspc.2023.105720.

[9] Shany Biton, Mohsin Aldhafeeri, Erez Marcusohn, Kenta Tsutsui, Tom Szwagier, Adi Elias, Julien Oster, Jean Marc Sellal, Mahmoud Suleiman, and Joachim A. Behar. Generalizable and robust deep learning algorithm for atrial fibrillation diagnosis across geography, ages and sexes. *npj Digital Medicine*, 6:44, 2023. doi: 10.1038/s41746-023-00791-1.

[10] Hang Zhang, Yanchun Zhao, Zhicheng Zhao, Xin Zhang, Zhanghao Guo, Yanni Chen, Jie Zheng, and Yuanyuan Li. Component-Aware Transformer for atrial fibrillation detection from short single-lead ECG. *IEEE Transactions on Biomedical Engineering*, 69(10):3201–3212, 2022. doi: 10.1109/TBME.2022.3161829.

[11] Lingwei Wang, Sibo Zhou, Yinlong Zhang, Chaolong Zou, Shan Zheng, and Wenguang Yang. MSGformer: A multi-scale grid transformer for 12-lead ECG classification. *IEEE Journal of Biomedical and Health Informatics*, 28(3):1456–1467, 2024. doi: 10.1109/JBHI.2023.3344523.

[12] Temesgen Mehari and Nils Strodthoff. Self-supervised representation learning from 12-lead ECG data. *Computers in Biology and Medicine*, 141:105114, 2022. doi: 10.1016/j.compbiomed.2021.105114.

[13] Yang Liu, Jun Pu, Jianying Wei, Long Zhao, Dan Zhao, Yumei Yue, Zheng Su, Changzheng Liu, and Lingqiao Wu. A Siamese Network for Efficient Personalized and Adaptive ECG Monitoring. *IEEE Transactions on Neural*

Systems and Rehabilitation Engineering, 31:2847–2856, 2023. doi: 10.1109/TNSRE.2023.3284199.

[14] Minje Choi, Edward Choi, and Hyunah Choi. WARN: Rethinking the Detection of Atrial Fibrillation from Temporal Resolution. *Patterns*, 5(4):100962, 2024. doi: 10.1016/j.patter.2024.100962.

[15] Ravi Kumar, Mohanasankar Sivaprakasam, and L. N. Sharma. Noise removal in ECG signals using wavelet decomposition based method. *Biomedical Signal Processing and Control*, 57:101780, 2020. doi: 10.1016/j.bspc.2019.101780.

[16] Prashant Singh and S. D. Joshi. Application of Empirical Wavelet Transform for Baseline Wander Removal in ECG Signals. In *Proceedings of the International Conference on Wireless Communications, Signal Processing and Networking (WiSPNET)*, pages 1326–1330, Chennai, India, 2017. doi: 10.1109/WiSPNET.2017.8299982.

[17] Huawen Chen, Taufiq Maharaj, and Salman Yousefi. DeScoD-ECG: Deep score-based diffusion model for ECG baseline wander and noise removal. *IEEE Journal of Biomedical and Health Informatics*, 27(7):3840–3851, 2023. doi: 10.1109/JBHI.2023.3247602.

[18] Filipe Miguel Dias et al. A biGRU denoiser for ECG signals. In *Computing in Cardiology*, Karlsruhe, Germany, 2024. doi: 10.22489/CinC.2024.289.

[19] Dongyang Zhang, Shuai Wang, Fang Li, Jian Wang, Arun Kumar Sangaiah, Victor S. Sheng, and Xiaoli Ding. An ECG Signal De-noising Approach Based on Wavelet Energy and Sub-band Smoothing Filter. *Applied Sciences*, 9(22):4968, 2019. doi: 10.3390/app9224968.

[20] Peng Wang, Beiping Hou, Sujie Shao, and Ruqiang Yan. ECG-based Personal Identification from Unsegmented Signals using Time Series Neural Network. *Biomedical Signal Processing and Control*, 85:104879, 2024. doi: 10.1016/j.bspc.2023.104879.

[21] Jungrae Kim et al. A Robust Wearable ECG Quality Indicator Using Robust Independent Component Analysis and Convolutional Neural Networks.

IEEE Transactions on Biomedical Engineering, 70(8):2401–2410, 2023. doi: 10.1109/TBME.2023.3244177.

[22] Jiapu Pan and Willis J. Tompkins. A real-time QRS detection algorithm. *IEEE Transactions on Biomedical Engineering*, BME-32(3):230–236, 1985. doi: 10.1109/TBME.1985.325532.

[23] Syed Anas Imtiaz and Esther Rodriguez-Villegas. Pan-Tompkins++: A Robust Real-Time QRS Detection Algorithm. *IEEE Access*, 11:18095–18111, 2023. doi: 10.1109/ACCESS.2023.3247979.

[24] Jaeseung Park, Sookyung Lee, and Moongu Jeon. An Adaptive and Time-Efficient ECG R-Peak Detection Algorithm. *Journal of Healthcare Engineering*, 2017:4528540, 2017. doi: 10.1155/2017/4528540.

[25] Xinge Liu, Hao Wang, Zhilin Li, and Lang Qin. A simple and effective deep neural network based QRS complex detection method on ECG signal. *Scientific Reports*, 14:17456, 2024. doi: 10.1038/s41598-024-68457-y.

[26] Muhammad Uzair Zahid, Serkan Kiranyaz, and Moncef Gabbouj. QRS detection in single-lead ECG signal using 1D-CNN. *Scientific Reports*, 12:12641, 2022. doi: 10.1038/s41598-022-16648-y.

[27] Yao Chen et al. A Multi-Label Classification Model for Heartbeat Signal. In *Computing in Cardiology*, Tampere, Finland, 2022. doi: 10.22489/CinC.2022.098.

[28] Jana Saclova, Ivo Viscor, and Marek Malik. Reliable P wave detection in pathological ECG signals. *Scientific Reports*, 12:6589, 2022. doi: 10.1038/s41598-022-10656-4.

[29] Alexander Petrov et al. U-Net based ECG Delineation with Classification-Guided Loss. In *Computing in Cardiology*, Atlanta, GA, USA, 2024. doi: 10.22489/CinC.2024.178.

[30] Preeti Madan, Vandana Singh, D. K. Singh, Arpit Dixit, and P. Srotriya. Hybrid deep learning approach for automatic arrhythmia detection using ECG. *Biomedical Signal Processing and Control*, 73:103396, 2022. doi: 10.1016/j.bspc.2021.103396.

[31] Sunil Kumar, Anshita Mallik, Arun Kumar, Javier Del Ser, and Guang Yang. Fuzz-ClustNet: Coupled fuzzy clustering and deep neural networks for arrhythmia detection from ECG signals. *Computers in Biology and Medicine*, 153:106511, 2023. doi: 10.1016/j.compbiomed.2022.106511.

[32] Jihyeon Han, Nishit Kamdar, et al. Novel Transformation Deep Learning Model for Electrocardiogram Classification and Arrhythmia Detection Using Edge Computing. *Journal of Grid Computing*, 22:3, 2024. doi: 10.1007/s10723-023-09702-y.

[33] Li Zhang et al. Efficient and Interpretable ECG Abnormality Detection via a Lightweight DSCR-BiGRU-Attention Network. *Mathematics*, 13(23):3882, 2024. doi: 10.3390/math13233882.

[34] Syed Bilal Ahmed, Irfan Hussain, Adnan Noor Qureshi, and Sohaib Ashraf. Hybrid Attention-Based Deep Learning Network (HADLN) for Automated ECG Classification. *EURASIP Journal on Advances in Signal Processing*, 2024:58, 2024. doi: 10.1186/s13634-024-01151-7.

[35] Kyle Johnson et al. ArrhythmiaVision: Resource-Conscious Deep Learning Models with Visual Explanations for ECG Arrhythmia Classification. arXiv preprint arXiv:2505.03787, 2024.

[36] Wenlong Li, Xiaoming Chen, Dawei Zha, Shiqiang Gu, and Zhigang Wang. An Atrial Fibrillation Detection Algorithm Based on Lightweight Design Architecture and Feature Fusion Strategy. *Biomedical Signal Processing and Control*, 91:105945, 2024. doi: 10.1016/j.bspc.2024.105945.

[37] Rintis Patel et al. ECG classification on embedded devices using asymmetric integer quantization. *IEEE Transactions on Biomedical Engineering*, 71(2): 634–643, 2024. doi: 10.1109/TBME.2023.3308419.

[38] Yunjun Zhao et al. Dynamic Range Quantization for ECG Classification on Wearable Devices. Presented at the Int. Conf. Wearable Technol., Boston, MA, USA, 2024.

[39] Ernesto Martinez-Rios, Pablo Nephtali Montes-Dorantes, and Jose A. Garcia-Macedo. A Novel Deep Learning Framework with a Pre-trained Model

as Feature Extractor for ECG Classification. *Computers in Biology and Medicine*, 168:107756, 2024. doi: 10.1016/j.compbiomed.2023.107756.

[40] Majid Sepahvand and Fardin Abdali-Mohammadi. A Novel Method for Reducing Arrhythmia Classification from 12-Lead ECG Signals to Single-Lead ECG with Minimal Loss of Accuracy through Teacher-Student Knowledge Distillation. *Information Sciences*, 593:64–77, 2022. doi: 10.1016/j.ins.2022.01.030.

[41] Yuren Liu and Yi-Ju Tseng. Deep Learning for ECG Classification: A Comparative Study of 1D and 2D Representations. *Biomedical Signal Processing and Control*, 92:106051, 2024. doi: 10.1016/j.bspc.2024.106051.

[42] Jan Novák et al. Comparative Study of Time-Frequency Transformation Methods for Electrocardiogram Classification. *Frontiers in Signal Processing*, 4:1322334, 2024. doi: 10.3389/frsip.2024.1322334.

[43] Alberto Rodriguez et al. Multi-Feature Fusion and Compressed Bi-LSTM for Memory-Efficient Heartbeat Classification. arXiv preprint arXiv:2405.15312, 2024.

[44] Li Chen et al. Mobile GPU Acceleration for 24-Hour ECG Analysis. *IEEE Transactions on Mobile Computing*, 23(5):2145–2158, 2024. doi: 10.1109/TMC.2024.3362014.

[45] D. Kim et al. High-Performance Computing for Large-Scale ECG Training. In *Computing in Cardiology*, Atlanta, GA, USA, 2024.

[46] Ary L. Goldberger, Luis A. N. Amaral, Leon Glass, Jeffrey M. Hausdorff, Plamen Ch. Ivanov, Roger G. Mark, Joseph E. Mietus, George B. Moody, Chung-Kang Peng, and H. Eugene Stanley. PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals. *Circulation*, 101(23):e215–e220, 2000. doi: 10.1161/01.CIR.101.23.e215.

[47] Patrick Wagner, Nils Strodthoff, Ralf-Dieter Bousseljot, Dieter Kreiseler, Fatima I. Lunze, Wojciech Samek, and Tobias Schaeffter. PTB-XL, a large pub-

licly available electrocardiography dataset. *Scientific Data*, 7:154, 2020. doi: 10.1038/s41597-020-0495-6.

[48] Nils Strodthoff, Patrick Wagner, Tobias Schaeffter, and Wojciech Samek. Deep Learning for ECG Analysis: Benchmarks and Insights from PTB-XL. *IEEE Journal of Biomedical and Health Informatics*, 25(5):1519–1528, 2021. doi: 10.1109/JBHI.2020.3022989.

[49] Erick A. Perez Alday, Annie Gu, Amit J. Shah, Chad Robichaux, An-Kwok Ian Wong, Chengyu Liu, Feifei Liu, Ali Bahrami Rad, Andoni Elola, Salman Seyedi, Qiao Li, Ashish Sharma, Gari D. Clifford, and Matthew A. Reyna. Classification of 12-lead ECGs: The PhysioNet/Computing in Cardiology Challenge 2020. *Physiological Measurement*, 41(12):124003, 2020. doi: 10.1088/1361-6579/abc960.

[50] Thomas De Brouwer et al. IRIDIA-AF: A wearable ECG database of patients with paroxysmal atrial fibrillation. In *Computing in Cardiology*, Atlanta, GA, USA, 2023. doi: 10.5281/zenodo.8405941.

[51] Kenta Tsutsui, Joachim A. Behar, Julien Oster, Takao Yoshida, Shigeki Kimura, Ryoma Yoshida, Ryozo Anan, Kenji Sunagawa, and Takafumi Emoto. SHDB-AF: A Japanese Ambulatory Holter ECG Database for Atrial Fibrillation Detection. *Scientific Data*, 11:892, 2024. doi: 10.1038/s41597-024-03756-3.

[52] Jae-Hyeon Kim et al. Cross-database evaluation of deep learning for atrial fibrillation detection. *IEEE Transactions on Biomedical Engineering*, 68(4): 1189–1198, 2021. doi: 10.1109/TBME.2020.3010421.

[53] Hang Zhang et al. AF-DETR: Transformer for Atrial Fibrillation Detection with Multi-Database Generalization. In *Computing in Cardiology*, Atlanta, GA, USA, 2024. doi: 10.22489/CinC.2024.214.

[54] Association for the Advancement of Medical Instrumentation. ANSI/AAMI EC57:2012/(R)2020 – Testing and reporting performance results of cardiac rhythm and ST segment measurement algorithms. AAMI, 2020.

[55] Jean O'Neill et al. Extended Holter Recording Using Novel Cardiac Monitor (Zio Patch) Changes Clinical Management of Patients with Cardiac Arrhythmias. *Stroke*, 50(5):1227–1232, 2019. doi: 10.1161/STROKEAHA.118.023190.

[56] Rolf Wachter, Marina Großmann, Kristina Keller, Deepak L. Bhatt, and Markus Zabel. Holter-ECG approach to atrial fibrillation detection in patients with cryptogenic stroke. *PLoS ONE*, 12(10):e0185386, 2017. doi: 10.1371/journal.pone.0185386.

[57] Michael A. Rosenberg, Minilu Samuel, Ami Thosani, and Peter J. Zimetbaum. Use of a Noninvasive Continuous Monitor to Reduce Hospitalizations and Cost in Patients With Heart Failure: A Randomised Controlled Trial. *International Journal of Cardiology*, 330:38–42, 2021. doi: 10.1016/j.ijcard.2021.02.059.

[58] Mintu P. Turakhia, David D. Hoang, Peter Zimetbaum, Joel D. Miller, Victor F. Froelicher, Uday N. Kumar, Xin Xu, Foula Yang, and Paul A. Heidenreich. Diagnostic utility of a novel leadless arrhythmia monitoring device. *The American Journal of Cardiology*, 112(4):520–524, 2013. doi: 10.1016/j.amjcard.2013.04.017.

[59] Emma Svennberg et al. Screening for Atrial Fibrillation in the General Population: The GUARD-AF Trial. In *ESC Congress*, London, UK, 2024.

[60] Marco V. Perez, Kenneth W. Mahaffey, Hazel Hedlin, John S. Rumsfeld, Alberto Garcia, Todd Ferris, Vidhya Balasubramanian, Andrea M. Russo, Abhinav Rajmane, Lauren Cheung, Grace Hung, Jacinta Lee, Peter Kowey, Nisha Talati, Divya Nag, Santosh E. Gummidipundi, Alexis Beatty, Mellanie T. Hills, Sumbul Desai, Christopher B. Granger, Deepak L. Bhatt, and Mintu P. Turakhia. Large-Scale Assessment of a Smartwatch to Identify Atrial Fibrillation. *New England Journal of Medicine*, 381(20):1909–1917, 2019. doi: 10.1056/NEJMoa1901183.

[61] Divya R. Seshadri et al. Accuracy of Apple Watch for Detection of Atrial Fibrillation. *Circulation*, 141(8):702–703, 2020. doi: 10.1161/CIRCULATIONAHA.119.044126.

[62] Joao Silva, Filipe Machado, Virginie Felizardo, Marcelo Gomes, Nuno M. Garcia, Eftim Zdravevski, Vitor Ramos, Plamen Savov, and Ivan Miguel Pires. Electrocardiogram Monitoring Wearable Devices and Artificial-Intelligence-Based Methods for Atrial Fibrillation Detection: A Review. *Sensors*, 20(6):1796, 2020. doi: 10.3390/s20061796.

[63] George B. Moody and Roger G. Mark. A new method for detecting atrial fibrillation using R-R intervals. *Computers in Cardiology*, 10:227–230, 1983. PhysioNet DOI: 10.13026/C2MW2D.

[64] George B. Moody. WFDB applications guide. PhysioNet, 2024. https://physionet.org/physiotools/wag/.

[65] Tim Bray. The JavaScript object notation (JSON) data interchange format. RFC 8259, 2017. https://www.rfc-editor.org/rfc/rfc8259.

[66] Apache Avro Project. Apache Avro specification. Apache Software Foundation, 2026. https://avro.apache.org/docs/current/specification/.

[67] The HDF Group. HDF5 field guide. HDF Group Documentation, 2026. https://support.hdfgroup.org/documentation/hdf5/latest/.

[68] Apache Parquet Project. Apache Parquet documentation. Apache Software Foundation, 2026. https://parquet.apache.org/docs/.

[69] Apache Spark Project. Parquet Files: Spark SQL data source guide. Apache Software Foundation, 2026. https://spark.apache.org/docs/latest/sql-data-sources-parquet.html.

[70] Google. Snappy: A fast compressor/decompressor. GitHub repository, 2025. https://github.com/google/snappy.

[71] Keisuke Tateno and Leon Glass. Automatic detection of atrial fibrillation using the coefficient of variation and density histograms of RR and delta RR intervals. *Medical and Biological Engineering and Computing*, 39(6):664–671, 2001. DOI: 10.1007/BF02345439.

[72] Douglas E. Lake and J. Randall Moorman. Accurate estimation of entropy in very short physiological time series: the problem of atrial fibrillation detection in implanted ventricular devices. *American Journal of Physiology-Heart and Circulatory Physiology*, 300(1):H319–H325, 2011. DOI: 10.1152/ajpheart.00561.2010.

[73] Andrius Petrėnas, Vaidotas Marozas, and Leif Sörnmo. Low-complexity detection of atrial fibrillation in continuous long-term monitoring. *Computers in Biology and Medicine*, 65:184–191, 2015. DOI: 10.1016/j.compbiomed.2015.01.019.

[74] Morteza Zabihi, A. B. Rad, Aggelos K. Katsaggelos, Serkan Kiranyaz, Susanna Narkilahti, and Moncef Gabbouj. Detection of atrial fibrillation in ECG hand-held devices using a random forest classifier. In *Computing in Cardiology 2017*, volume 44, pages 1–4, 2017. DOI: 10.22489/CinC.2017.069-336.

[75] Sebastian D. Goodfellow, Andrew Goodwin, Robert Greer, Peter C. Laussen, Mjaye Mazwi, and Danny Eytan. Atrial fibrillation classification using step-by-step machine learning. *Biomedical Physics & Engineering Express*, 4(4): 045005, 2018. DOI: 10.1088/2057-1976/aabef4.

[76] Zhaohan Xiong, Martin K. Stiles, and Jichao Zhao. Robust ECG signal classification for detection of atrial fibrillation using a novel neural network. In *Computing in Cardiology 2017*, volume 44, pages 1–4, 2017. DOI: 10.22489/CinC.2017.066-138.

[77] Martin Zihlmann, Dmytro Perekrestenko, and Michael Tschannen. Convolutional recurrent neural networks for electrocardiogram classification. In *Computing in Cardiology 2017*, volume 44, pages 1–4, 2017. DOI: 10.22489/CinC.2017.070-060.

[78] Qince Liu et al. CPSC 2021: Paroxysmal atrial fibrillation event detection from dynamic ECG recordings. PhysioNet, 2021. DOI: 10.13026/cpsc2021.